

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО «СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
ИНСТИТУТ ЦИФРОВОГО РАЗВИТИЯ
МЕЖИНСТИТУТСКАЯ БАЗОВАЯ КАФЕДРА

КУРСОВОЙ ПРОЕКТ

по дисциплине

«Технология разработки программного обеспечения»

на тему:

«Разработка веб-приложения для бронирования билетов»

Выполнил:

Щугарев Данила Михайлович
студент 2 курса группы ПИЖ-б-о-24-1
направления подготовки 09.03.04
«Программная инженерия»
направленность (профиль)
«Разработка и сопровождение
программного обеспечения»
очной формы обучения

(подпись)

Руководитель работы:
Ф.В. Самойлов, доцент
межинститутской базовой кафедры

Работа допущена к защите

(подпись руководителя)

(дата)

Работа выполнена и
защищена с оценкой

Дата защиты _____

Председатель комиссии:

зав. межинститутской
базовой кафедрой

(подпись) Е. Н. Новикова

Члены комиссии:

доцент МИБК

(подпись) Ф.В. Самойлов

доцент МИБК

(подпись) И.В. Свистунов

Ставрополь, 2026 г.

Министерство науки и высшего образования и Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии

Кафедра межинститутская базовая

Направление подготовки 09.03.04 «Программная инженерия»

Направленность (профиль) «Разработка и сопровождение программного обеспечения»

ЗАДАНИЕ

на курсовой проект

студента Щугарева Данилы Михайловича

(фамилия, имя, отчество)

по дисциплине «Технология разработки программного обеспечения»

1. Тема проекта Разработка веб-приложения для бронирования билетов

2. Цель: Повышение уровня профессиональной подготовки путем углубления и закрепления теоретических знаний и практических навыков, приобретенных в результате изучения дисциплины «Технология разработки программного обеспечения», а также формирование компетенций в области проектирования, разработки и документирования современных веб-приложений с использованием фреймворка Django в рамках выбранной траектории выполнения.

3. Задачи:

3.1. Анализ прикладной задачи и методов её решения

- Провести предпроектное обследование предметной области.
- Выполнить анализ бизнес-процессов и сформулировать требования к программному обеспечению.
- Обосновать выбор технологического стека в соответствии с выбранной траекторией:
- Траектория Б: Django, Django REST Framework, React, JWT, PostgreSQL.
- Проанализировать существующие аналоги и обосновать необходимость разработки.

3.2. Разработка алгоритмов решения задачи

- Разработать модель предметной области (Domain Model).
- Спроектировать Use Case диаграмму и спецификации прецедентов.
- Разработать ER-диаграмму базы данных.
- Спроектировать архитектуру приложения в соответствии с выбранной траекторией:

- Трассировка Б: клиент-серверная архитектура с REST API.
- Разработать алгоритмы основных сценариев (регистрация, аутентификация, CRUD-операции).

3.3. Реализация программного кода

- Реализовать модели данных с учётом связей между таблицами.
- Реализовать систему аутентификации:
- Трассировка Б: JWT-аутентификация (access/refresh токены).
- Реализовать бизнес-логику:
- Трассировка Б: REST API (ViewSet, Serializer) и React-приложение.
- Реализовать дополнительные функции согласно выбранной трассировки:
- Трассировка Б: полноценное SPA с маршрутизацией и управлением состоянием.

3.4. Отладка и тестирование программного кода

- Провести модульное тестирование ключевых компонентов (модели, API, компоненты React).
- Выполнить интеграционное тестирование взаимодействия между слоями.
- Провести системное тестирование сценариев использования.
- Зафиксировать результаты тестирования в отчёте.

4. Перечень подлежащих разработке вопросов:

- а) теоретической части: изучение и анализ литературы, постановка условия задачи, выбор и описание методов и библиотек для ее решения и технологий, среды программирования
- б) проектная часть: проектирование UML-диаграмм классов, разработка алгоритмов решения задачи, методов классов
- в) реализация: описание структуры проекта, описание файлов проекта, разработка программного кода, тестирование программы

5. Исходные данные:

- а) по литературным источникам: ГОСТы, международные стандарты, программные средства, используемые при разработке программного обеспечения
- б) исходные данные, подготовленные для тестирования объекта профессиональной деятельности (информационной системы / приложения / программного продукта)

6. Список рекомендуемой литературы: монографии, диссертации, научные статьи, учебно-методические материалы, ссылки на официальные сайты, содержащие информацию необходимую для решения поставленных в работе задач

1. Кузьмин, А. Г. Разработка фронтенд-приложений / А. Г. Кузьмин. — Санкт-Петербург : Питер, 2025. — 496 с. — (Библиотека программиста). — ISBN 978-5-4461-4272-9.

2. Кухтин, С. А. Frontend-разработка сайтов и приложений на HTML, CSS, JavaScript и React / С. А. Кухтин. — Санкт-Петербург : Наука и Техника, 2026. — 512 с. — ISBN 978-5-907592-84-1.
3. Орбайсета, А. С. Создание фронтенд-фреймворка с нуля / А. С. Орбайсета. — Санкт-Петербург : Питер, 2025. — 384 с. — (Библиотека программиста). — ISBN 978-5-4461-4201-9.
4. Курс «Фронтенд-разработчик» от Яндекс Практикум. — URL: <https://practicum.yandex.ru/frontend-developer/>.
5. Проектирование и разработка WEB-приложений. Введение в frontend и backend разработку на JavaScript и node.js : учебное пособие для вузов. — 4-е изд., стер. — Санкт-Петербург : Лань, 2025. — 120 с. — ISBN 978-5-507-51011-5
6. Чернышев, С. А. Основы Flutter / С. А. Чернышев, Ю. М. Петров, С. П. Ильин, П. А. Гершевич. — Санкт-Петербург : Питер, 2026. — 688 с. — (Библиотека программиста). — ISBN 978-5-4461-4469-3.
7. Григорьев В. В. Python и Django. Разработка веб-приложений. — СПб.: Питер, 2024. — 512 с.
8. Дронов В. А. Django 5. Практика создания веб-сайтов. — СПб.: БХВ-Петербург, 2024. — 544 с.
9. Ломов А. Ю. Веб-разработка на Django 5. — М.: ДМК Пресс, 2025. — 412 с.
10. Документация Django. URL: <https://docs.djangoproject.com/>
11. Документация Django REST Framework. URL: <https://www.django-rest-framework.org/>
12. Django REST Framework Simple JWT Documentation. URL: <https://django-rest-framework-simplejwt.readthedocs.io/>
13. Документация React. URL: <https://react.dev/>
14. Документация React Router. URL: <https://reactrouter.com/>
15. Документация Axios. URL: <https://axios-http.com/>
16. Документация SQLite. URL: <https://sqlite.org/>
17. Документация JWT. URL: <https://jwt.io/>
18. Документация Visual Studio Code. URL: <https://code.visualstudio.com/>
19. Документация Git. URL: <https://git-scm.com/>

7. Контрольные сроки представления отдельных разделов курсовой работы:

- 25 % - предоставление первого раздела «12» марта 2026 г.
- 50 % - предоставление второго раздела «09» апреля 2026 г.
- 75 % - предоставление третьего раздела «30» апреля 2026 г.
- 100 % - предоставление работы на отзыв «14» мая 2026 г.

8. Срок защиты студентом курсовой работы «23» мая 2026 г.

Дата выдачи задания «12» февраля 2026 г.

Руководитель курсового проекта

К.т.н.
(ученая степень, звание)

(личная подпись)

Ф.В. Самойлов
(инициалы, фамилия)

Задание принял к исполнению студент очной формы обучения 2 курса группы

ПИЖ-б-о-24-1

(личная подпись)

Д.М. Щугарев
(инициалы, фамилия)

ПАСПОРТ ПРОЕКТА

Таблица 1 - Паспорт проекта

Параметр	Значение
Название проекта	Разработка веб-приложения для бронирования билетов
Исполнитель	Щугарев Данила Михайлович, группа ПИЖ-б-о-24-1
Руководитель	Самойлов Филипп Владимирович, к.т.н., доцент
Дисциплина	Технология разработки программного обеспечения
Траектория	Б — Django REST Framework + React SPA
Тема	Сайт для бронирования билетов
Технологический стек	Backend: Django, Django REST Framework, Simple JWT SQLite. Frontend: React, React Router, Axios
Основные модели	User, Profile, EventCategory, Event, Booking, Comment
Тип аутентификации	JWT (access-токен 30 мин., refresh-токен 1 день)
Количество API эндпоинтов	8+
Количество React- компонентов	5+
Период разработки	Февраль 2026 — Май 2026
Репозиторий	Два репозитория: backend/ и frontend/ (или монорепоизиторий)
Максимальная оценка	5 (отлично)

Содержание

ВВЕДЕНИЕ	11
Актуальность темы исследования	11
Цель и задачи работы	12
Объект и предмет исследования	13
1. АНАЛИТИЧЕСКАЯ ЧАСТЬ	15
1.1. Описание предметной области	15
1.2. Анализ бизнес-процессов (IDEF0)	18
1.3. SWOT-анализ текущего состояния	23
1.4 Анализ аналогов и конкурентов	24
1.5 Обоснование необходимости разработки	26
1.6 Экономическое обоснование (ROI)	27
2. ПРОЕКТНАЯ ЧАСТЬ	29
2.1. Модель требований к ПО	29
2.1.1. Use Case диаграмма	29
2.1.2. Спецификация прецедентов	32
2.1.3. Глоссарий терминов	33
2.2 Модель предметной области	34
2.2.1. Domain Model (диаграмма классов)	34
2.2.2. Описание сущностей и атрибутов	35
2.2.3. Бизнес-правила	36
2.3. Архитектурное проектирование	37
2.3.1. Выбор архитектурного стиля	37
2.3.2. Диаграмма компонентов (Django + React)	37
2.3.3. Описание слоёв и их ответственности	38
2.3.4. Схема взаимодействия (REST API + JWT)	39
2.4. Проектирование базы данных	41
2.4.1. ER-диаграмма	41
2.4.2. Физическая модель данных	42
2.4.3. DDL-скрипты	43
2.5. Детальное проектирование	44

2.5.1. Диаграммы последовательности (аутентификация, CRUD)	44
2.5.2 Диаграммы классов проектирования	46
2.5.3. Применение паттернов проектирования	47
3. РЕАЛИЗАЦИОННАЯ ЧАСТЬ	48
3.1. Реализация бизнес-логики	48
3.1.1. Структура проекта Django	48
3.1.2 Модели-сущности (Category, Article, User)	49
3.1.3 Слой доступа к данным (ORM Django)	50
3.1.4 Слой управления (ViewSet, Serializer)	51
3.2 Реализация API	52
3.2.1 REST API эндпоинты (8+)	52
3.2.2 JWT-аутентификация (access/refresh токены)	53
3.2.3 Разрешения (permissions) для разграничения доступа	54
3.2.4 Документация API (Swagger/OpenAPI)	55
3.3 Реализация фронтенда (React)	55
3.3.1 Структура проекта React	56
3.3.2 Компоненты (ArticleList, ArticleDetail, Login, Register)	57
3.3.3 Маршрутизация (React Router)	58
3.3.4 Управление состоянием (AuthContext)	59
3.3.5 Взаимодействие с API (Axios, interceptors)	59
3.4 Рефакторинг и оптимизация	60
3.4.1 Статический анализ кода (flake8, ESLint)	61
3.4.2 Оптимизация запросов (select_related, prefetch_related)	61
3.4.3 Кэширование на клиенте (React Query / RTK Query)	62
3.5 Безопасность и транзакции	63
3.5.1 JWT-аутентификация	63
3.5.2 Защита от XSS, CSRF (CORS настройки)	64
3.5.3 Валидация данных на сервере и клиенте	64
5. РАЗВЁРТЫВАНИЕ И ЭКСПЛУАТАЦИЯ	65
5.1. Контейнеризация (Docker)	65
5.2 Инструкция по установке	65
5.3 Инструкция по настройке (.env, CORS)	66

5.4 Требования к окружению (Python, Node.js, PostgreSQL)	68
6. УПРАВЛЕНИЕ ПРОЕКТОМ	69
6.1 WBS (иерархическая структура работ)	69
6.2 Диаграмма Ганта	69
6.3 Оценка трудозатрат (COCOMO)	70
6.4 Управление рисками	71
ЗАКЛЮЧЕНИЕ	71
Выводы	71
Результаты	72
Перспективы развития	72
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	73
Приложения	74
Приложение А. Листинги кода (models.py, serializers.py, consumers.py, React components)	74
Приложение Б. Скриншоты интерфейсов	89

ВВЕДЕНИЕ

Актуальность темы исследования

В настоящее время информационные технологии активно внедряются во все сферы деятельности человека, включая индустрию развлечений и организации массовых мероприятий. Концерты, спортивные соревнования, театральные постановки, выставки и другие события требуют удобных и надежных средств продажи и бронирования билетов. Традиционные способы приобретения билетов постепенно уступают место электронным системам, обеспечивающим быстрый доступ к информации о мероприятиях и возможность дистанционного оформления заказа.

Развитие веб-технологий и широкое распространение сети Интернет способствуют росту популярности онлайн-сервисов бронирования билетов. Использование современных веб-приложений позволяет пользователям получать актуальную информацию о мероприятиях, выбирать подходящие места, оформлять бронирование и управлять своими заказами в любое время и из любого места.

С точки зрения разработчиков программного обеспечения особый интерес представляют современные архитектурные подходы к созданию веб-приложений. Одним из наиболее распространенных решений является разделение клиентской и серверной частей приложения с использованием REST API и одностраничных приложений (SPA). Такой подход обеспечивает высокую масштабируемость, гибкость разработки и возможность независимого развития фронтенда и бэкенда.

В связи с этим разработка веб-приложения для бронирования билетов на основе технологий Django REST Framework и React является актуальной задачей, позволяющей реализовать современную клиент-серверную архитектуру, обеспечить безопасную аутентификацию пользователей и предоставить удобный интерфейс взаимодействия с системой.

Цель и задачи работы

Целью курсового проекта является автоматизация процессов поиска мероприятий, бронирования билетов и управления пользовательскими данными путем разработки веб-приложения, обеспечивающего пользователям удобный доступ к информации о мероприятиях, возможность дистанционного оформления бронирований и эффективное взаимодействие с системой через единый программный интерфейс.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Выполнить анализ предметной области системы бронирования билетов.
2. Провести исследование существующих аналогов и определить их преимущества и недостатки.
3. Сформировать функциональные и нефункциональные требования к разрабатываемой системе.
4. Спроектировать архитектуру клиент-серверного приложения.
5. Разработать структуру базы данных и определить связи между сущностями предметной области.
6. Реализовать серверную часть приложения с использованием Django REST Framework.
7. Реализовать REST API для взаимодействия между клиентской и серверной частями системы.
8. Реализовать JWT-аутентификацию пользователей на основе access и refresh токенов.
9. Настроить разграничение прав доступа для различных категорий пользователей.
10. Разработать клиентскую часть приложения с использованием React SPA.
11. Реализовать маршрутизацию и управление состоянием приложения.
12. Реализовать механизм автоматического обновления токенов доступа.
13. Провести тестирование разработанного программного обеспечения.
14. Подготовить комплект проектной документации.

Объект и предмет исследования

Объектом исследования являются процессы организации и автоматизации бронирования билетов на мероприятия в сети Интернет.

Предметом исследования выступают методы и технологии разработки современных веб-приложений, основанных на использовании REST API, SPA-архитектуры и механизмов безопасной аутентификации пользователей.

Методы исследования

При выполнении курсового проекта использовались следующие методы исследования:

- анализ предметной области;
- моделирование бизнес-процессов;
- объектно-ориентированное проектирование;
- проектирование баз данных;
- разработка клиент-серверных приложений;
- тестирование программного обеспечения.

В процессе разработки были использованы современные технологии и программные средства: язык программирования Python, фреймворк Django REST Framework, библиотека React, язык JavaScript, система управления базами данных SQLite, а также библиотека Simple JWT для реализации механизма аутентификации пользователей.

Практическая значимость работы

Практическая значимость работы заключается в создании функционирующего веб-приложения для бронирования билетов, которое может использоваться для организации продажи билетов на различные мероприятия. Разработанное приложение обеспечивает удобный интерфейс взаимодействия пользователей с системой, безопасную аутентификацию и разграничение прав доступа, а также предоставляет

инструменты управления мероприятиями и бронированиями.

Разработанная система построена на основе современной клиент-серверной архитектуры с разделением фронтенда и бэкенда, что обеспечивает возможность дальнейшего масштабирования и расширения функциональных возможностей приложения.

1. АНАЛИТИЧЕСКАЯ ЧАСТЬ

1.1. Описание предметной области

Предметная область разрабатываемой системы относится к сфере автоматизации процессов продажи и бронирования билетов на различные мероприятия. В настоящее время цифровые сервисы становятся основным способом приобретения билетов благодаря удобству использования, высокой скорости обработки заказов и возможности удалённого взаимодействия пользователей с системой.

Под мероприятием понимается организованное событие культурного, развлекательного или спортивного характера, доступное для посещения пользователями. К таким событиям относятся концерты, театральные постановки, выставки, фестивали, спортивные соревнования и иные массовые мероприятия. Каждое мероприятие характеризуется набором параметров: названием, описанием, категорией, датой проведения, местом проведения, стоимостью билета и количеством доступных мест.

Традиционные способы продажи билетов через кассы или сторонние сервисы обладают рядом недостатков, среди которых ограниченное время работы, отсутствие оперативного доступа к информации о мероприятиях и сложность управления заказами. Использование веб-приложений позволяет устранить указанные недостатки и обеспечить пользователям круглосуточный доступ к сервису бронирования.

Разрабатываемая система представляет собой веб-приложение, построенное на основе клиент-серверной архитектуры с разделением фронтенда и бэкенда. Серверная часть реализована с использованием фреймворка Django REST Framework и предоставляет REST API для обработки запросов и хранения данных. Клиентская часть разработана с использованием библиотеки React и реализована в виде одностраничного приложения (SPA), обеспечивающего

удобный пользовательский интерфейс.

В системе предусмотрены следующие категории пользователей:

1. Гость - неавторизованный пользователь, имеющий возможность просматривать список мероприятий и получать информацию о них без регистрации в системе.
2. Авторизованный пользователь - зарегистрированный пользователь, обладающий возможностью бронирования билетов, управления своими бронированиями, создания комментариев и взаимодействия с системой в полном объеме, предусмотренном пользовательскими правами.
3. Администратор - пользователь с расширенными правами доступа, осуществляющий управление категориями мероприятий, модерацию контента и контроль работы системы.

В ходе анализа предметной области были выделены основные сущности системы:

- User - пользователь системы;
- Profile - профиль пользователя;
- EventCategory - категория мероприятия;
- Event - мероприятие;
- Booking - бронирование билета;
- Comment - комментарий пользователя.

Сущность «Пользователь» содержит информацию об учетной записи и правах доступа. Для хранения дополнительных данных используется сущность «Профиль пользователя». Сущность «Категория мероприятия» предназначена для классификации событий по типам. Сущность «Мероприятие» содержит сведения о проводимых событиях, включая дату проведения, место, стоимость билетов и количество доступных мест. Сущность «Бронирование» фиксирует факт оформления пользователем заказа на мероприятие. Сущность «Комментарий» обеспечивает возможность обратной связи и обсуждения мероприятий пользователями.

Функционирование системы предполагает следующий сценарий

взаимодействия. Пользователь просматривает доступные мероприятия, при необходимости проходит регистрацию и авторизацию, после чего получает доступ к функционалу бронирования билетов и управления своими данными. Администратор осуществляет управление категориями и контроль информации, размещённой в системе.

Таким образом, предметная область характеризуется необходимостью автоматизации процессов публикации мероприятий, бронирования билетов и управления пользовательскими данными в рамках единой информационной системы.

Таблица 2 - Матрица стейкхолдеров

Стейкхолдер	Роль	Интересы	Влияние	Ожидания
Гость	Потенциальный пользователь	Получение информации о мероприятиях	Низкое	Удобный просмотр каталога
Пользователь	Конечный пользователь	Бронирование билетов и управление профилем	Высокое	Удобный интерфейс и безопасность
Администратор	Управление системой	Контроль контента и пользователей	Очень высокое	Полный доступ к данным
Разработчик	Поддержка и развитие системы	Чистая архитектура и расширяемость	Среднее	Поддерживаемый программный код
Руководитель проекта	Контроль качества	Соответствие требованиям	Очень высокое	Полная документация проекта

Матрица стейкхолдеров позволяет определить заинтересованные стороны проекта, их влияние на процесс разработки и ожидания от разрабатываемой системы.

1.2. Анализ бизнес-процессов (IDEF0)

В рамках предметной области разрабатываемого веб-приложения ключевым является бизнес-процесс, связанный с организацией поиска мероприятий и бронирования билетов пользователями через сеть Интернет. Для формализации и описания функционирования системы используется методология IDEF0, позволяющая представить бизнес-процессы через входные данные, управляющие воздействия, механизмы и результаты работы системы.

Основной бизнес-процесс системы может быть определён как «Функционирование веб-приложения для бронирования билетов» и представлен в виде контекстной диаграммы уровня А-0.

Входные данные

К входным данным системы относятся:

1. Запросы пользователей на просмотр мероприятий;
2. Данные о мероприятиях;
3. Регистрационные данные пользователей;
4. Информация о бронированиях;
5. Комментарии пользователей.

Управляющие воздействия

Функционирование системы регулируется следующими управляющими воздействиями:

1. Правила работы веб-приложения;
2. Бизнес-логика обработки бронирований;
3. Правила аутентификации и авторизации;
4. Политика разграничения доступа пользователей;
5. Ограничения и правила валидации данных.

Механизмы

Для реализации бизнес-процессов используются следующие механизмы:

1. Django REST Framework;
2. React SPA;
3. REST API;
4. СУБД SQLite;
5. Пользователь;
6. Администратор системы.

Выходные данные

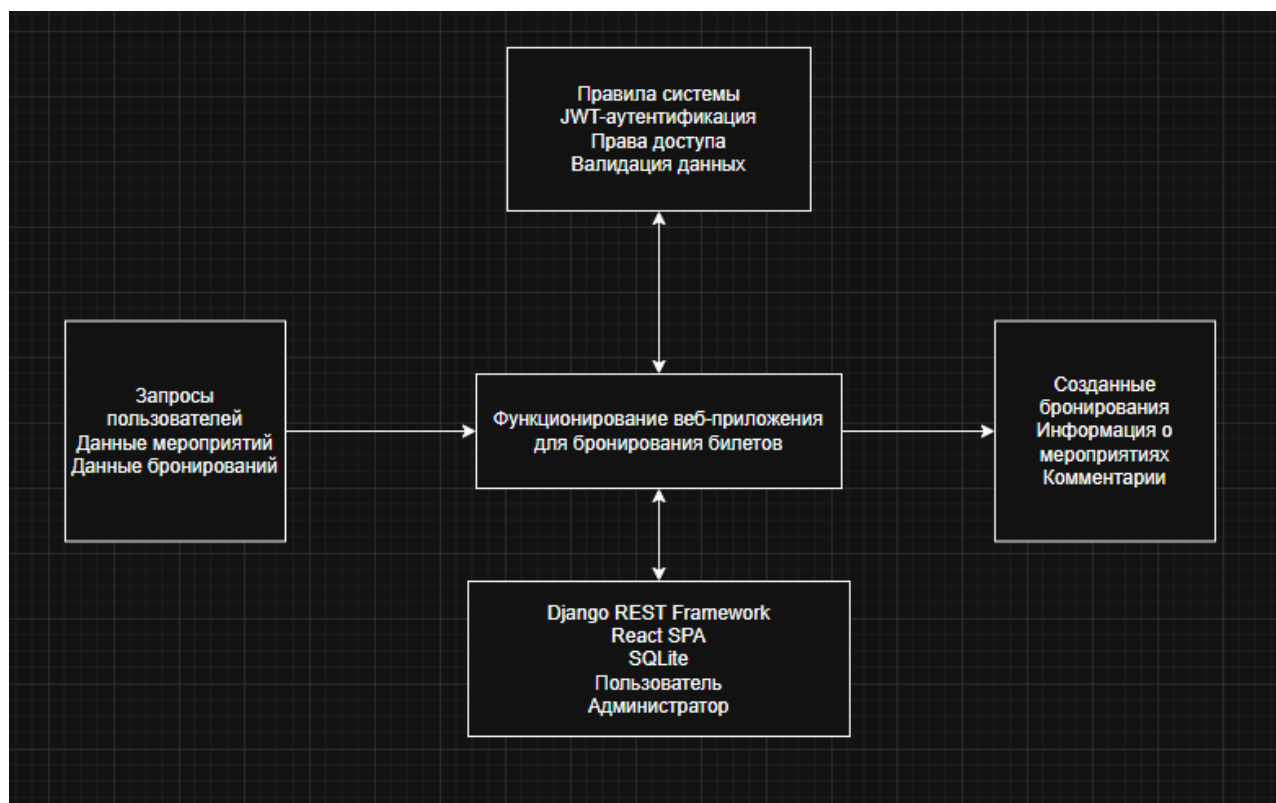
Результатом работы системы являются:

1. Созданные бронирования;
2. Обновлённые данные о мероприятиях;
3. Информация для пользователей;
4. Комментарии и отзывы;
5. Данные пользовательских профилей.

Контекстная диаграмма A-0

Контекстная диаграмма уровня A-0 отражает взаимодействие системы с внешней средой. В качестве входов выступают действия пользователей и данные о мероприятиях. Управляющими воздействиями являются правила работы системы и механизмы безопасности. В качестве механизмов используются программные средства и пользователи системы. Результатом функционирования выступают созданные бронирования и предоставление информации пользователям.

Рисунок 1 - Контекстная диаграмма IDEF0 уровня А-0



Декомпозиция бизнес-процесса А0

Для более детального описания функционирования системы была выполнена декомпозиция бизнес-процесса уровня А0 на отдельные подпроцессы.

В состав декомпозиции входят следующие работы:

А1. Управление мероприятиями

Подпроцесс включает:

- создание мероприятий;
- редактирование информации о мероприятиях;
- удаление мероприятий;
- управление категориями.

Исполнителями данного процесса являются авторизованные пользователи и администраторы.

А2. Регистрация и аутентификация пользователей

Данный процесс обеспечивает:

- регистрацию новых пользователей;
- вход в систему;
- получение JWT-токенов;
- обновление access-токенов посредством refresh-токенов;
- выход из системы.

Для обеспечения безопасности используется механизм JWT-аутентификации.

А3. Просмотр мероприятий

Подпроцесс включает:

- просмотр списка мероприятий;
- поиск мероприятий;
- фильтрацию по категориям;
- просмотр подробной информации о мероприятии.

Данный функционал доступен как гостям, так и авторизованным пользователям.

А4. Бронирование билетов

В рамках данного процесса пользователь:

- выбирает мероприятие;
- выполняет бронирование билета;
- просматривает историю бронирований.

Информация о бронированиях сохраняется в базе данных и становится доступной в личном кабинете пользователя.

А5. Работа с комментариями

Подпроцесс предусматривает:

- добавление комментариев;
- просмотр комментариев других пользователей;

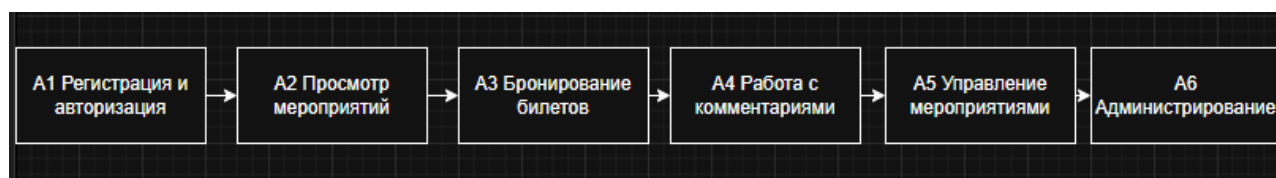
- удаление комментариев при наличии соответствующих прав доступа.

А6. Администрирование системы

Администратор выполняет:

- управление пользователями;
- управление категориями;
- контроль данных системы;
- модерацию контента.

Рисунок 2 - Диаграмма декомпозиции А0 бизнес-процесса функционирования веб-приложения для бронирования билетов



Описание процесса функционирования системы

Работа пользователя с системой начинается с просмотра списка доступных мероприятий. Для получения расширенного функционала пользователь проходит регистрацию и авторизацию. После успешного входа в систему пользователю становятся доступны функции бронирования билетов и управления личными данными.

При создании бронирования система проверяет права доступа пользователя, наличие свободных билетов и корректность введенных данных. После успешного завершения операции информация сохраняется в базе данных.

Администратор системы осуществляет управление категориями мероприятий и контролирует корректность данных, размещенных в системе.

Проведенный анализ бизнес-процессов позволил формализовать функционирование разрабатываемой системы, определить ключевые операции и установить взаимодействие между пользователями и программным обеспечением. Полученные результаты используются на последующих этапах проектирования архитектуры и реализации веб-приложения.

1.3. SWOT-анализ текущего состояния

Для оценки перспектив разработки веб-приложения для бронирования билетов проведём SWOT-анализ, позволяющий выявить сильные и слабые стороны системы, а также определить внешние возможности и угрозы, способные повлиять на её дальнейшее развитие и эксплуатацию.

В качестве текущего состояния рассматривается существующий процесс приобретения билетов через различные интернет-сервисы и традиционные кассы без использования единой специализированной системы, разработанной в рамках данного проекта.

Таблица 3 - SWOT-анализ разрабатываемого веб-приложения

Сильные стороны (Strengths)	Слабые стороны (Weaknesses)
Использование современной клиент-серверной архитектуры	Необходимость поддержки двух частей системы (backend и frontend)
Применение Django REST Framework и React	Более высокая сложность разработки SPA-приложений
Безопасная JWT-аутентификация пользователей	Необходимость настройки механизмов безопасности
Разграничение прав доступа пользователей	Дополнительные затраты на тестирование
Масштабируемость и расширяемость системы	Повышенные требования к квалификации разработчиков
Удобный пользовательский интерфейс	Зависимость от стабильности интернет-соединения
Возможности (Opportunities)	Угрозы (Threats)
Добавление онлайн-оплаты билетов	Возможные кибератаки и попытки несанкционированного доступа

Сильные стороны (Strengths)	Слабые стороны (Weaknesses)
Интеграция со сторонними сервисами	Утечка пользовательских данных
Разработка мобильного приложения	Рост нагрузки на сервер при увеличении числа пользователей
Расширение функциональности системы	Технические сбои программного обеспечения
Использование аналитики пользовательского поведения	Конкуренция со стороны существующих сервисов

Проведённый SWOT-анализ показывает, что разработка веб-приложения для бронирования билетов обладает значительным потенциалом развития благодаря использованию современных технологий веб-разработки и гибкой архитектуры системы. Использование REST API и SPA-подхода обеспечивает высокую масштабируемость и возможность дальнейшего расширения функциональности приложения.

Одновременно с этим необходимо уделить особое внимание вопросам безопасности данных пользователей, защите API и обеспечению стабильной работы системы при увеличении нагрузки.

Таким образом, результаты SWOT-анализа подтверждают целесообразность разработки веб-приложения и демонстрируют перспективы его дальнейшего развития.

1.4 Анализ аналогов и конкурентов

Перед разработкой программного обеспечения был проведён анализ существующих сервисов бронирования билетов, представленных на рынке. Цель анализа заключается в выявлении преимуществ и недостатков существующих решений, а также определении возможностей

повышения качества разрабатываемой системы.

В качестве объектов анализа были выбраны популярные сервисы продажи и бронирования билетов.

Таблица 4 - Сравнительный анализ аналогов

Система	Основной функционал	Преимущества	Недостатки
Яндекс Афиша	Просмотр мероприятий и покупка билетов	Большой каталог мероприятий, удобный интерфейс	Часть функций зависит от внешних сервисов
Kassir.ru	Продажа билетов на мероприятия	Широкая география использования	Сложная навигация по отдельным разделам
Ticketland	Онлайн-продажа билетов	Популярность и развитая инфраструктура	Необходимость регистрации для полного функционала
Разрабатываемая система	Просмотр мероприятий, бронирование билетов, комментарии	Простота использования, современная архитектура, гибкость развития	Ограниченный функционал учебного проекта

Существующие сервисы обладают широкими функциональными возможностями и используются большим количеством пользователей. Однако большинство подобных систем являются крупными коммерческими решениями, ориентированными на массовый рынок и обладающими избыточным функционалом для решения отдельных

специализированных задач.

Разрабатываемое веб-приложение отличается использованием современной архитектуры Django REST Framework и React SPA, а также применением JWT-аутентификации для защиты пользовательских данных. Разделение клиентской и серверной частей приложения позволяет независимо развивать отдельные компоненты системы и повышать её масштабируемость.

Кроме того, система предоставляет различные уровни доступа для гостей, авторизованных пользователей и администраторов, что обеспечивает гибкое управление функциональностью и безопасность данных.

Проведённый анализ аналогов показывает, что разрабатываемое приложение способно обеспечить базовый функционал современных систем бронирования билетов и обладает возможностью дальнейшего расширения.

1.5 Обоснование необходимости разработки

В настоящее время наблюдается устойчивый рост популярности онлайн-сервисов продажи и бронирования билетов. Пользователи всё чаще предпочитают приобретать билеты через интернет, что позволяет экономить время и получать доступ к актуальной информации о мероприятиях независимо от местоположения.

Несмотря на наличие большого количества существующих сервисов, разработка собственного веб-приложения остаётся актуальной задачей. Это обусловлено необходимостью изучения современных технологий веб-разработки и создания программного продукта, реализующего современные подходы к проектированию клиент-серверных систем.

Разрабатываемая система позволяет автоматизировать процессы поиска мероприятий, регистрации пользователей и бронирования билетов. Использование REST API обеспечивает независимость клиентской и серверной

частей приложения, а применение React SPA позволяет создавать удобный и интерактивный пользовательский интерфейс.

Дополнительным преимуществом является использование JWT-аутентификации, обеспечивающей безопасное хранение и передачу данных пользователей.

Таким образом, разработка веб-приложения для бронирования билетов является актуальной и практически значимой задачей, направленной на автоматизацию процессов взаимодействия пользователей с сервисом и применение современных технологий разработки программного обеспечения.

1.6 Экономическое обоснование (ROI)

Для оценки экономической эффективности разработки программного продукта используется показатель ROI (Return on Investment), характеризующий возврат инвестиций.

В рамках учебного проекта основными затратами являются трудозатраты разработчика, связанные с проектированием, программированием, тестированием и документированием системы.

Таблица 5 - Оценка трудозатрат

Этап разработки	Трудозатраты, часы
Анализ предметной области	12
Проектирование системы	20
Разработка backend	45
Разработка frontend	40
Тестирование	15

Этап разработки	Трудозатраты, часы
Подготовка документации	18
Итого	150

Предположим среднюю стоимость одного часа работы разработчика равной 500 рублей.

Тогда общая стоимость разработки составит:

75000 руб.

Предположим, что внедрение системы позволит сократить временные затраты на обработку заявок и бронирований, что эквивалентно экономическому эффекту в размере 100 000 рублей.

Полученное значение показывает, что разработка системы является экономически целесообразной и обладает положительным экономическим эффектом.

ВЫВОД

В рамках аналитической части была исследована предметная область системы бронирования билетов, проведён анализ бизнес-процессов и существующих аналогов, выполнен SWOT-анализ и экономическое обоснование разработки.

Результаты проведённого анализа подтвердили актуальность и практическую значимость разработки веб-приложения для бронирования билетов, а также сформировали основу для последующего проектирования и реализации программного обеспечения.

2. ПРОЕКТНАЯ ЧАСТЬ

2.1. Модель требований к ПО

Модель требований к программному обеспечению определяет функциональные возможности системы, состав пользователей и сценарии их взаимодействия с приложением. Формализация требований позволяет определить границы системы и обеспечить соответствие разрабатываемого программного продукта поставленным целям.

Разрабатываемое веб-приложение предназначено для автоматизации процесса бронирования билетов на мероприятия и обеспечивает взаимодействие пользователей с системой посредством веб-интерфейса.

Основными категориями пользователей являются:

- гость;
- авторизованный пользователь;
- администратор.

Каждая категория пользователей обладает определённым набором прав доступа и выполняет различные операции в системе.

2.1.1. Use Case диаграмма

Для описания функциональных возможностей системы была разработана диаграмма вариантов использования (Use Case Diagram), отражающая взаимодействие пользователей с веб-приложением.

В системе выделяются три основных актора:

- Гость - неавторизованный пользователь системы;
- Пользователь - зарегистрированный пользователь;
- Администратор - пользователь с расширенными правами доступа.

Гостям доступны операции просмотра мероприятий, регистрации и авторизации.

Авторизованные пользователи могут просматривать мероприятия, бронировать билеты, оставлять комментарии и управлять своими данными.

Администратор обладает дополнительными возможностями по управлению категориями и содержимым системы.

Рисунок 3 – Use Case диаграмма веб-приложения для бронирования билетов



2.1.2. Спецификация прецедентов

Прецедент UC-1 «Регистрация пользователя»

Характеристика	Описание
Актор	Гость
Цель	Создание учётной записи
Предусловие	Пользователь не авторизован
Основной сценарий	Ввод данных → отправка формы → создание аккаунта
Результат	Новый пользователь зарегистрирован

Прецедент UC-2 «Авторизация пользователя»

Характеристика	Описание
Актор	Пользователь
Цель	Получение доступа к системе
Предусловие	Наличие учётной записи
Основной сценарий	Ввод логина и пароля → получение JWT-токенов
Результат	Пользователь авторизован

Прецедент UC-3 «Бронирование билета»

Характеристика	Описание
Актор	Авторизованный пользователь
Цель	Создание бронирования
Предусловие	Пользователь вошёл в систему
Основной сценарий	Выбор мероприятия → бронирование билета
Результат	Создана запись о бронировании

Прецедент UC-4 «Управление мероприятиями»

Характеристика	Описание
Актор	Пользователь, администратор
Цель	Управление мероприятиями
Предусловие	Наличие прав доступа
Результат	Данные мероприятия изменены

Прецедент UC-5 «Управление категориями»

Характеристика	Описание
Актор	Администратор
Цель	Управление категориями
Предусловие	Пользователь является администратором
Результат	Категории обновлены

2.1.3. Глоссарий терминов

Таблица 6 - Глоссарий терминов

Термин	Определение
Пользователь	Зарегистрированный участник системы
Гость	Неавторизованный пользователь
Администратор	Пользователь с расширенными правами доступа
Мероприятие	Событие, доступное для бронирования
Категория	Тип мероприятия
Бронирование	Процесс резервирования билета
REST API	Интерфейс взаимодействия между клиентом и сервером
JWT	Технология токеной аутентификации
Access Token	Токен доступа к защищённым ресурсам
Refresh Token	Токен обновления доступа

Термин	Определение
SPA	Одностраничное веб-приложение
React Router	Библиотека маршрутизации React
Axios	Библиотека HTTP-запросов
Serializer	Средство преобразования объектов в JSON
ViewSet	Компонент обработки API-запросов
ORM	Технология объектно-реляционного отображения

2.2 Модель предметной области

Модель предметной области описывает основные сущности системы, их атрибуты и взаимосвязи. Построение модели предметной области позволяет формализовать структуру данных и определить правила взаимодействия объектов системы.

В разработанном веб-приложении основными сущностями являются пользователи, мероприятия, категории мероприятий, бронирования и комментарии.

2.2.1. Domain Model (диаграмма классов)

Для описания структуры системы была разработана диаграмма классов (Domain Model), отражающая основные сущности предметной области и связи между ними.

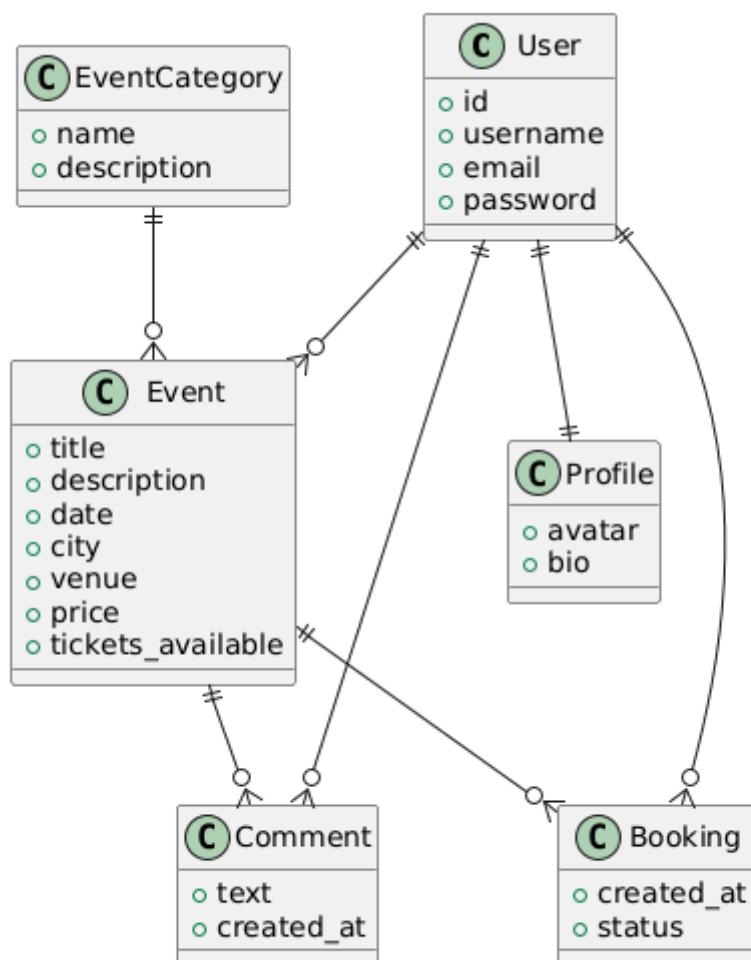
В системе выделяются следующие сущности:

- User — пользователь системы;
- Profile — профиль пользователя;
- EventCategory — категория мероприятий;
- Event — мероприятие;
- Booking — бронирование;

- Comment — комментарий.

Связи между сущностями реализованы посредством механизмов Django ORM и внешних ключей базы данных.

Рисунок 4 - Domain Model веб-приложения для бронирования билетов



2.2.2. Описание сущностей и атрибутов

Основные сущности системы и их ключевые атрибуты представлены в таблице.

Таблица 6 - Сущности и атрибуты системы

Сущность	Основные атрибуты
User	username, email, password

Profile	avatar, bio
EventCategory	name, description
Event	title, description, date, price
Booking	user, event, created_at
Comment	text, created_at

Сущность User предназначена для хранения информации о зарегистрированных пользователях системы.

Сущность Profile содержит дополнительную информацию о пользователе.

Сущность EventCategory используется для классификации мероприятий.

Сущность Event хранит информацию о мероприятии, включая дату проведения, описание и стоимость билета.

Сущность Booking обеспечивает хранение информации о бронировании билетов пользователями.

Сущность Comment предназначена для хранения пользовательских комментариев и отзывов.

2.2.3. Бизнес-правила

Функционирование системы определяется набором бизнес-правил, обеспечивающих корректность работы приложения и безопасность данных.

Основные бизнес-правила системы:

1. Только зарегистрированный пользователь может создавать бронирования.
2. Гость имеет доступ только к просмотру мероприятий и регистрации.
3. Авторизованный пользователь может оставлять комментарии.
4. Администратор обладает расширенными правами управления системой.
5. Доступ к защищённым API-эндпоинтам осуществляется только при наличии JWT-токена.
6. Каждое бронирование связано с конкретным пользователем и мероприятием.
7. Мероприятие относится только к одной категории.

8. Удаление связанных объектов выполняется в соответствии с правилами Django ORM.

Соблюдение указанных бизнес-правил обеспечивает целостность данных и корректное функционирование разработанного программного продукта.

2.3. Архитектурное проектирование

Архитектурное проектирование является одним из ключевых этапов разработки программного обеспечения, определяющим структуру системы, способы взаимодействия её компонентов и механизмы обработки данных.

При разработке веб-приложения для бронирования билетов была выбрана современная клиент-серверная архитектура, обеспечивающая разделение ответственности между серверной и клиентской частями приложения. Такой подход повышает масштабируемость системы, упрощает сопровождение программного кода и позволяет независимо развивать отдельные компоненты приложения.

2.3.1. Выбор архитектурного стиля

В качестве архитектурного стиля была выбрана клиент-серверная архитектура.

Серверная часть реализована на основе Django REST Framework и отвечает за хранение данных, обработку запросов и реализацию бизнес-логики.

Клиентская часть реализована с использованием React и обеспечивает взаимодействие пользователя с системой посредством веб-интерфейса.

Обмен данными между компонентами осуществляется через REST API с использованием формата JSON.

Выбранная архитектура обеспечивает:

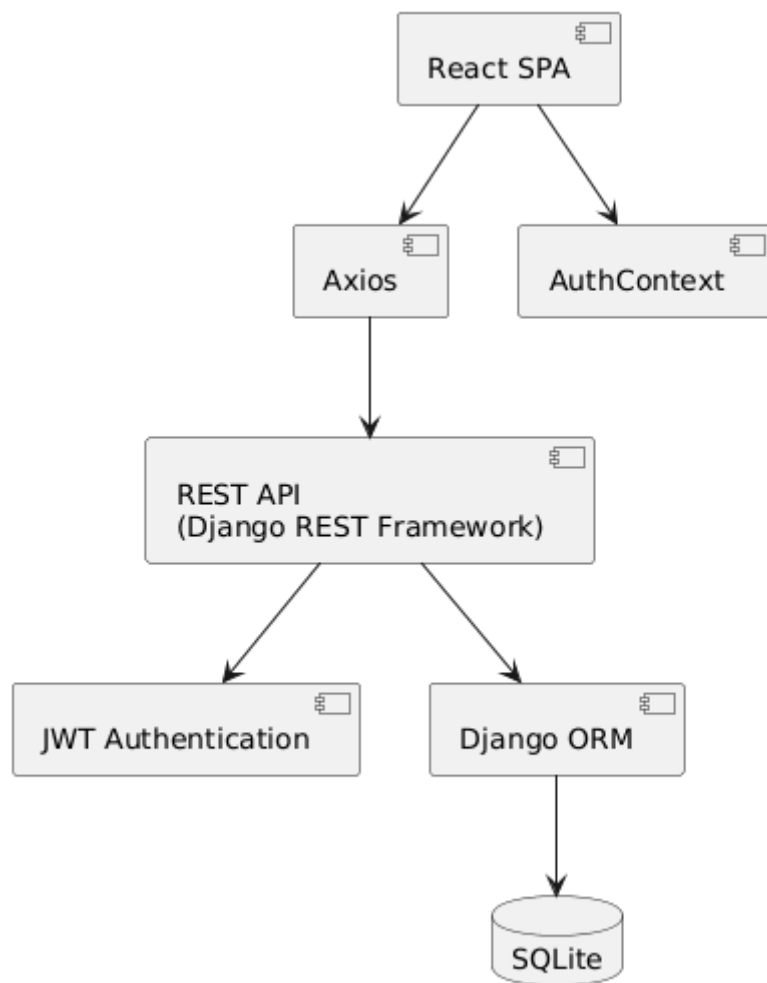
- независимую разработку frontend и backend;
- возможность масштабирования системы;
- удобство сопровождения программного продукта;
- поддержку различных клиентских приложений.

2.3.2. Диаграмма компонентов (Django + React)

Основными компонентами системы являются React-приложение, REST

API, механизм аутентификации и база данных.

Рисунок 5 - Диаграмма компонентов системы



На клиентской стороне выполняется отображение пользовательского интерфейса и отправка запросов к серверу.

Серверная часть выполняет обработку запросов, взаимодействие с базой данных и проверку прав доступа пользователей.

2.3.3. Описание слоёв и их ответственности

Архитектура приложения включает несколько логических слоёв, каждый из которых выполняет определённые функции.

Таблица 7 - Слои системы и их назначение

Слой	Назначение
React SPA	Пользовательский интерфейс
REST API	Обработка HTTP-запросов
Django ORM	Работа с базой данных
SQLite	Хранение данных
JWT	Аутентификация и авторизация

Клиентский слой обеспечивает взаимодействие пользователя с системой.

Слой REST API реализует бизнес-логику приложения и обеспечивает обмен данными между клиентом и сервером.

Слой ORM выполняет преобразование объектов Python в записи базы данных.

СУБД SQLite используется для хранения информации о пользователях, мероприятиях и бронированиях.

2.3.4. Схема взаимодействия (REST API + JWT)

Взаимодействие между компонентами системы осуществляется посредством REST API и механизма JWT-аутентификации.

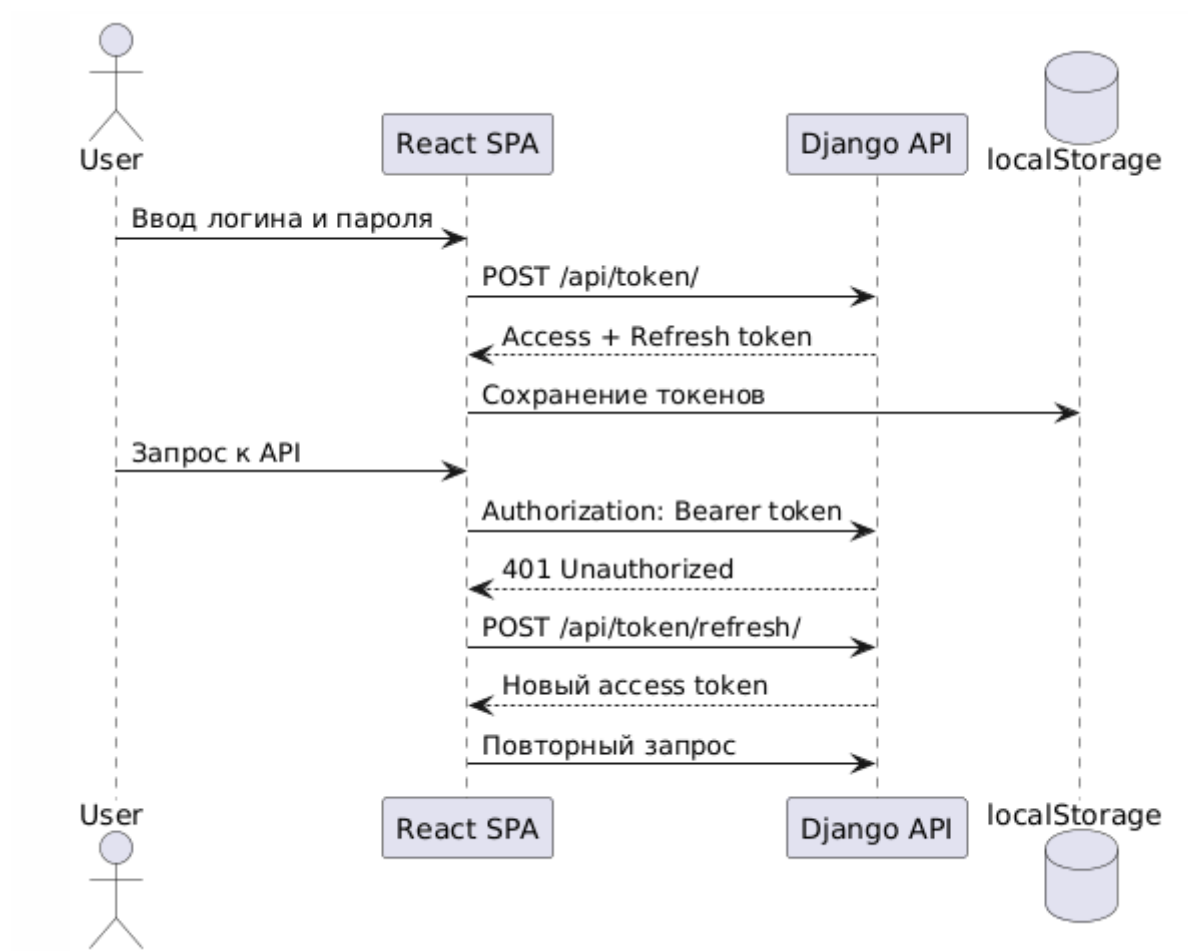
При авторизации пользователь получает access и refresh токены, которые используются для доступа к защищённым ресурсам системы.

Access token передаётся в заголовке HTTP-запроса:

Authorization: Bearer <access_token>

При истечении срока действия access token клиент автоматически получает новый token через механизм обновления.

Рисунок 6 - Схема взаимодействия REST API и JWT-аутентификации



2.4. Проектирование базы данных

Проектирование базы данных является одним из важнейших этапов разработки программного обеспечения, поскольку от структуры хранения данных зависит производительность, надёжность и расширяемость системы.

Для хранения информации в разрабатываемом веб-приложении используется реляционная база данных SQLite, интегрированная с фреймворком Django посредством ORM (Object-Relational Mapping).

Выбор SQLite обусловлен следующими преимуществами:

- простота настройки и использования;
- отсутствие необходимости отдельного сервера базы данных;
- интеграция с Django по умолчанию;
- достаточная производительность для учебных проектов;
- поддержка стандартного языка SQL.

Проектируемая база данных предназначена для хранения информации о пользователях, мероприятиях, бронированиях и комментариях.

2.4.1. ER-диаграмма

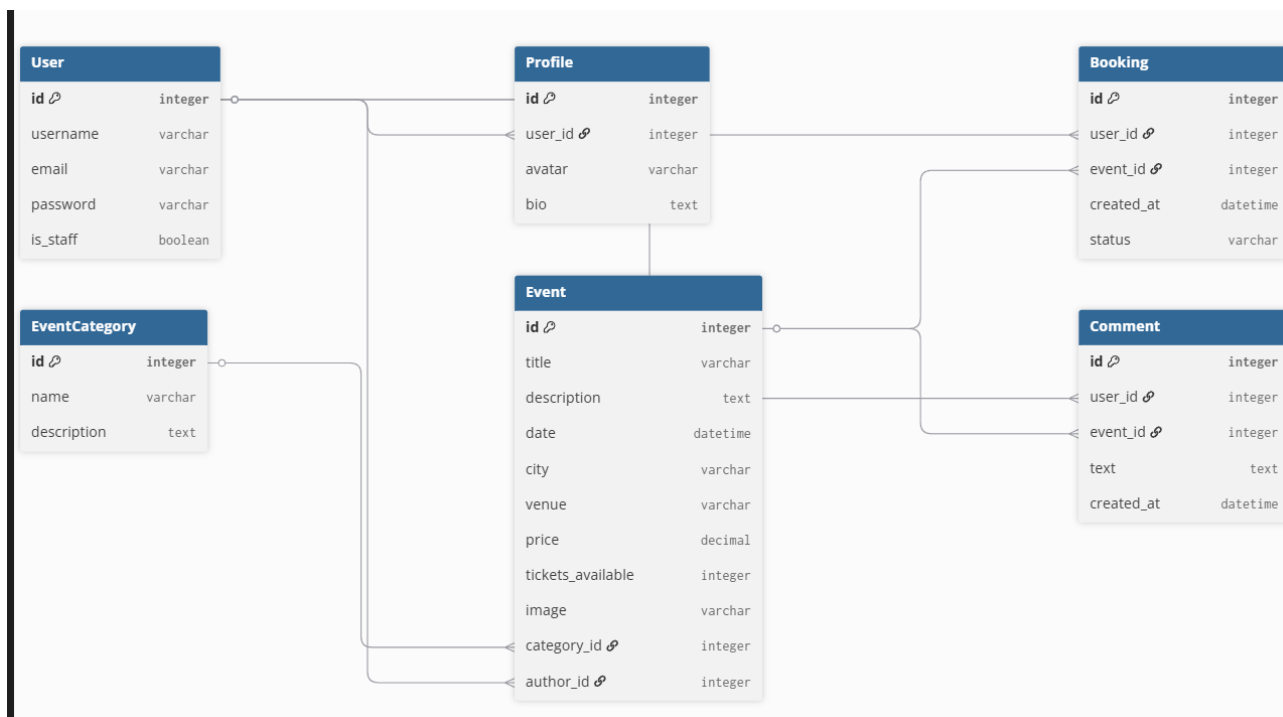
Для описания структуры базы данных была построена ER-диаграмма (Entity Relationship Diagram), отражающая сущности системы и связи между ними.

Основными сущностями являются:

- User;
- Profile;
- EventCategory;
- Event;
- Booking;
- Comment.

Связи между сущностями реализуются посредством внешних ключей и обеспечивают целостность данных.

Рисунок 7 - ER-диаграмма базы данных веб-приложения



2.4.2. Физическая модель данных

Физическая модель данных определяет набор таблиц базы данных и их назначение.

Таблица 8 – Таблицы базы данных

Таблица	Назначение
User	Хранение данных пользователей
Profile	Дополнительная информация о пользователях
EventCategory	Хранение категорий мероприятий
Event	Хранение информации о мероприятиях
Booking	Хранение данных о бронированиях
Comment	Хранение комментариев пользователей

Таблица User содержит сведения о зарегистрированных пользователях системы.

Таблица Profile хранит дополнительную информацию о пользователях.

Таблица EventCategory используется для классификации мероприятий.

Таблица Event содержит сведения о мероприятиях, включая описание,

дату проведения и стоимость билетов.

Таблица Booking хранит информацию о бронировании билетов пользователями.

Таблица Comment обеспечивает хранение комментариев и отзывов пользователей.

Использование реляционной модели данных обеспечивает целостность информации и поддерживает эффективную работу приложения.

2.4.3. DDL-скрипты

Создание таблиц базы данных выполняется автоматически средствами Django ORM посредством механизма миграций.

Для генерации структуры базы данных используются команды:

```
python manage.py makemigrations
```

```
python manage.py migrate
```

Ниже приведён пример SQL-структуры таблицы мероприятий:

```
CREATE TABLE Event (  
    id INTEGER PRIMARY KEY,  
    title VARCHAR(255),  
    description TEXT,  
    date DATETIME,  
    price DECIMAL(10,2),  
    category_id INTEGER  
);
```

Создание остальных таблиц выполняется аналогичным образом автоматически на основании моделей Django.

Использование миграций позволяет отслеживать изменения структуры базы данных и обеспечивает переносимость проекта между различными средами выполнения.

2.5. Детальное проектирование

Детальное проектирование является завершающим этапом проектирования программного обеспечения и направлено на описание взаимодействия компонентов системы, структуры классов и применяемых архитектурных решений.

На данном этапе формализуются сценарии работы пользователей с системой, определяется структура программных компонентов и выбираются шаблоны проектирования, обеспечивающие гибкость и расширяемость приложения.

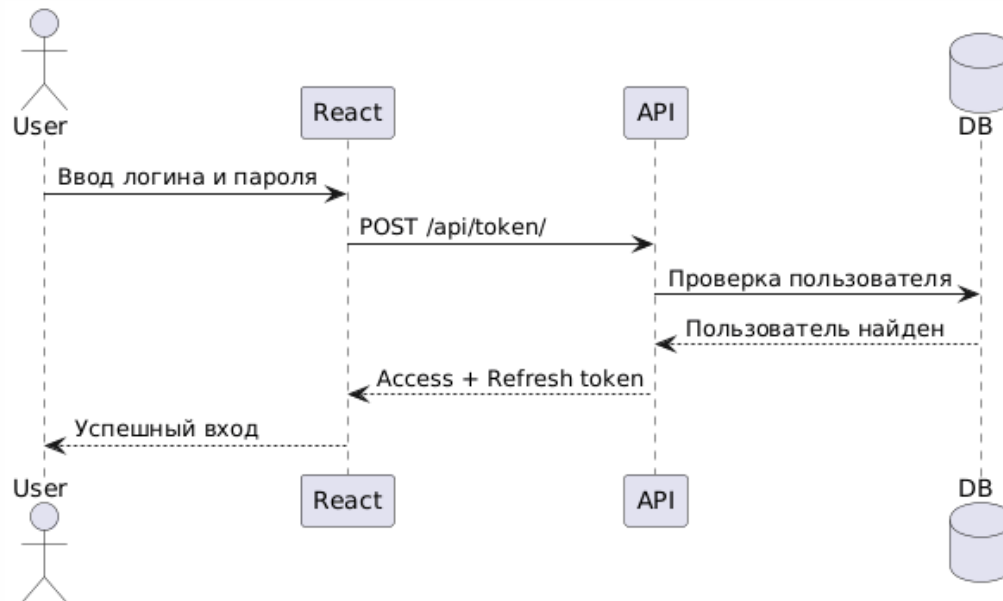
2.5.1. Диаграммы последовательности (аутентификация, CRUD)

Диаграммы последовательности позволяют описать взаимодействие компонентов системы во времени и определить порядок выполнения операций. Для разработанного веб-приложения были построены диаграммы последовательности основных сценариев работы системы.

Первая диаграмма описывает процесс авторизации пользователя:

1. Пользователь вводит логин и пароль.
2. React-приложение отправляет запрос на сервер.
3. Django REST API выполняет проверку учётных данных.
4. При успешной проверке пользователю выдаются JWT-токены.
5. Токены сохраняются в браузере и используются для дальнейшей работы.

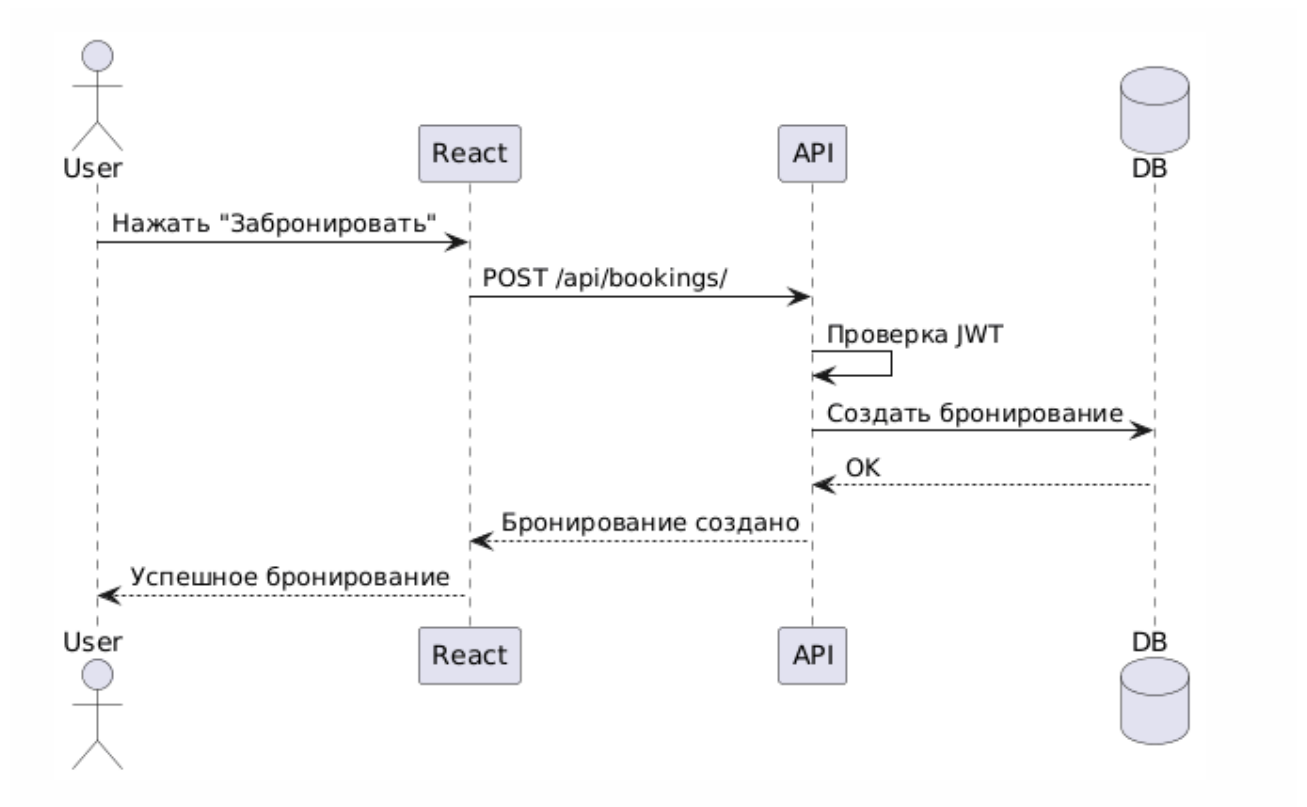
Рисунок 8 - Диаграмма последовательности авторизации пользователя



Вторая диаграмма описывает процесс бронирования билета:

1. Пользователь выбирает мероприятие.
2. Клиентское приложение отправляет запрос на сервер.
3. Сервер проверяет права доступа пользователя.
4. Выполняется создание записи о бронировании.
5. Пользователь получает подтверждение операции.

Рисунок 9 - Диаграмма последовательности бронирования билета



2.5.2 Диаграммы классов проектирования

При разработке программного обеспечения использовался объектно-ориентированный подход, основанный на разделении системы на независимые классы и модули.

Основными классами системы являются:

- User;
- Profile;
- EventCategory;
- Event;
- Booking;
- Comment.

Связи между классами реализованы посредством механизмов Django ORM и отражают структуру предметной области.

2.5.3. Применение паттернов проектирования

В процессе разработки программного обеспечения использовались архитектурные решения и шаблоны проектирования, обеспечивающие гибкость и сопровождаемость программного продукта.

Основные применяемые паттерны представлены в таблице.

Таблица 9 – Используемые паттерны проектирования

Паттерн	Назначение
Model-View-Template (MVT)	Архитектура Django-приложения
Serializer	Преобразование объектов в JSON
ViewSet	Реализация REST API
Context API	Управление состоянием React
React Hooks	Управление состоянием компонентов
Interceptor	Автоматическое обновление JWT-токенов

Паттерн MVT используется во фреймворке Django и обеспечивает разделение логики приложения на модели, представления и шаблоны.

Паттерн Serializer применяется для преобразования объектов моделей в формат JSON и обратно.

Паттерн ViewSet позволяет реализовать CRUD-операции с минимальным количеством кода.

Механизм Context API используется для хранения данных авторизованного пользователя и управления состоянием приложения.

Использование Axios Interceptor обеспечивает автоматическое обновление access token при истечении срока его действия.

Применение перечисленных архитектурных решений способствует повышению качества программного обеспечения и упрощает его сопровождение.

3. РЕАЛИЗАЦИОННАЯ ЧАСТЬ

3.1. Реализация бизнес-логики

Серверная часть веб-приложения для бронирования билетов реализована с использованием фреймворка Django и библиотеки Django REST Framework. Выбор данных технологий обусловлен их высокой производительностью, развитой экосистемой и удобством разработки REST API.

Основными задачами серверной части являются:

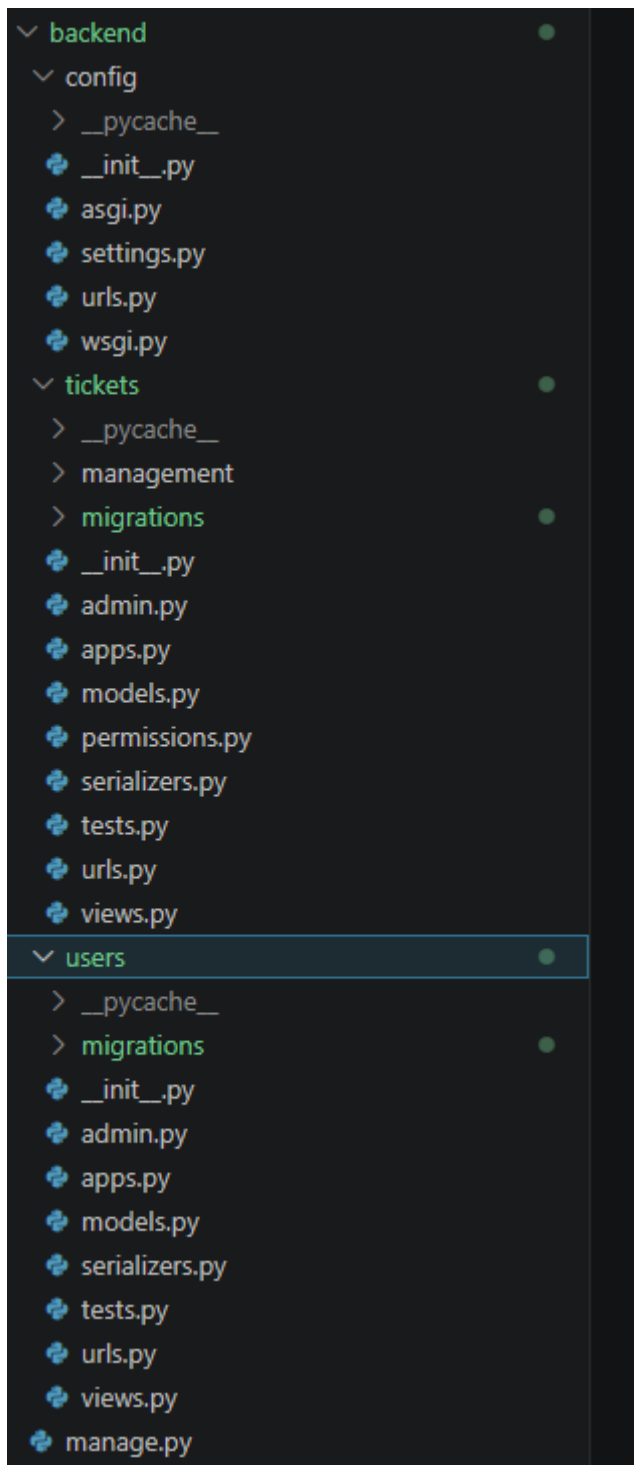
- хранение данных;
- реализация бизнес-логики;
- обработка HTTP-запросов;
- аутентификация и авторизация пользователей;
- разграничение прав доступа;
- взаимодействие с базой данных.

Для хранения информации используется база данных SQLite, интегрированная с Django посредством ORM.

3.1.1. Структура проекта Django

Проект организован по модульному принципу и состоит из нескольких приложений, каждое из которых отвечает за определённую функциональность системы.

Рисунок 10 – структура проекта Django



Файл `settings.py` содержит настройки проекта, включая параметры базы данных, установленные приложения и настройки безопасности. Файл `urls.py` отвечает за маршрутизацию запросов. Приложения системы реализуют отдельные функциональные модули.

3.1.2 Модели-сущности (Category, Article, User)

В соответствии с тематикой проекта роль сущности `Category` выполняет

модель категории мероприятий, роль Article — модель мероприятия, а роль User — модель пользователя системы.

Основными моделями являются:

- User — пользователь системы;
- EventCategory — категория мероприятий;
- Event — мероприятие;
- Booking — бронирование;
- Comment — комментарий.

Модель пользователя обеспечивает регистрацию, аутентификацию и хранение информации о пользователях.

Модель категории предназначена для классификации мероприятий по типам.

Модель мероприятия содержит сведения о названии, описании, дате проведения, месте проведения и стоимости билетов.

Модель бронирования хранит информацию о приобретённых билетах и их владельцах.

Модель комментариев обеспечивает обратную связь пользователей.

Листинг 1 – Пример модели мероприятия

```
class Event(models.Model):  
    title = models.CharField(max_length=255)  
    description = models.TextField()  
    date = models.DateTimeField()  
    price = models.DecimalField(max_digits=10, decimal_places=2)  
    category = models.ForeignKey(EventCategory,  
                                on_delete=models.CASCADE)
```

3.1.3 Слой доступа к данным (ORM Django)

Для работы с базой данных используется технология Django ORM (Object Relational Mapping), обеспечивающая преобразование объектов Python в записи реляционной базы данных.

ORM позволяет выполнять операции создания, чтения, изменения и

удаления объектов без написания SQL-запросов.

Примеры использования ORM:

```
events = Event.objects.all()
```

```
booking = Booking.objects.create(  
    user=request.user,  
    event=event  
)
```

```
comments = Comment.objects.filter(  
    event=event  
)
```

Использование ORM повышает переносимость приложения и упрощает сопровождение программного кода.

3.1.4 Слой управления (ViewSet, Serializer)

Для обработки API-запросов используются ViewSet и Serializer из библиотеки Django REST Framework.

Сериализаторы предназначены для преобразования объектов моделей в формат JSON и обратно.

Пример сериализатора:

```
class EventSerializer(serializers.ModelSerializer):  
    class Meta:  
        model = Event  
        fields = "__all__"
```

ViewSet обеспечивает обработку HTTP-запросов и выполнение CRUD-операций.

Пример ViewSet:

```
class EventViewSet(ModelViewSet):  
    queryset = Event.objects.all()
```

```
serializer_class = EventSerializer
```

Использование ViewSet и Serializer позволяет сократить объём программного кода и упростить разработку REST API.

3.2 Реализация API

Для обеспечения взаимодействия клиентской и серверной частей приложения был реализован программный интерфейс REST API на основе Django REST Framework. Передача данных между компонентами системы осуществляется посредством HTTP-запросов с использованием формата JSON.

Реализованный API обеспечивает выполнение операций создания, получения, изменения и удаления данных, а также поддерживает механизмы аутентификации и разграничения прав доступа.

3.2.1 REST API эндпоинты (8+)

В разработанном приложении реализован набор REST API эндпоинтов, обеспечивающих взаимодействие клиентской части с сервером.

Таблица 10 - Основные REST API эндпоинты

Метод	URL	Назначение
GET	/api/events/	Получение списка мероприятий
GET	/api/events/{id}/	Получение информации о мероприятии
POST	/api/events/	Создание мероприятия
PUT	/api/events/{id}/	Редактирование мероприятия
DELETE	/api/events/{id}/	Удаление мероприятия
GET	/api/categories/	Получение категорий
POST	/api/bookings/	Создание бронирования
GET	/api/bookings/	Получение списка бронирований
POST	/api/comments/	Добавление комментария

POST	/api/token/	Получение JWT-токенов
POST	/api/token/refresh/	Обновление access token

Каждый эндпоинт поддерживает соответствующие HTTP-методы и выполняет операции над ресурсами системы в соответствии с принципами REST-архитектуры.

Взаимодействие между клиентом и сервером осуществляется посредством HTTP-протокола, а данные передаются в формате JSON.

3.2.2 JWT-аутентификация (access/refresh токены)

Для обеспечения безопасности доступа к API в системе реализована JWT-аутентификация с использованием библиотеки Simple JWT.

После успешной авторизации сервер формирует два токена:

- Access Token — используется для доступа к защищённым ресурсам API;
- Refresh Token — используется для получения нового access token после истечения срока его действия.

Процесс аутентификации выполняется следующим образом:

1. Пользователь вводит логин и пароль.
2. Клиент отправляет запрос на сервер.
3. Сервер проверяет учётные данные пользователя.
4. При успешной авторизации сервер возвращает access и refresh токены.
5. Токены сохраняются в localStorage браузера.
6. При обращении к защищённым ресурсам access token передаётся в заголовке Authorization.

Пример заголовка авторизации:

Authorization: Bearer <access_token>

При истечении срока действия access token клиент автоматически выполняет запрос на обновление токена через эндпоинт /api/token/refresh/.

Использование JWT позволяет отказаться от хранения серверных сессий и обеспечивает масштабируемость приложения.

3.2.3 Разрешения (permissions) для разграничения доступа

Для обеспечения безопасности системы реализован механизм разграничения прав доступа пользователей.

В приложении выделены три категории пользователей:

1. Гость - неавторизованный пользователь.
2. Авторизованный пользователь.
3. Администратор.

Гостям доступны следующие операции:

- просмотр мероприятий;
- просмотр информации о мероприятиях;
- регистрация и авторизация.

Авторизованные пользователи дополнительно могут:

- бронировать билеты;
- создавать комментарии;
- управлять собственными мероприятиями;
- просматривать историю бронирований.

Администраторы обладают расширенными правами доступа:

- управление категориями;
- управление пользователями;
- удаление объектов системы;
- полный доступ к данным приложения.

Для реализации ограничений доступа используются классы разрешений Django REST Framework:

```
permission_classes = [IsAuthenticated]
```

а также пользовательские разрешения при необходимости.

Использование механизма permissions обеспечивает защиту данных и предотвращает несанкционированный доступ к ресурсам системы.

3.2.4 Документация API (Swagger/OpenAPI)

Документирование REST API является важной частью разработки современных веб-приложений.

В рамках проекта описание API выполнялось средствами Django REST Framework и тестированием эндпоинтов через браузер и инструменты разработчика.

При дальнейшем развитии проекта возможно использование спецификации OpenAPI и инструмента Swagger для автоматической генерации документации API.

Использование Swagger позволяет:

- просматривать список эндпоинтов;
- тестировать API непосредственно из браузера;
- получать информацию о параметрах запросов и ответов;
- автоматически документировать программный интерфейс.

Наличие документации API значительно упрощает сопровождение и развитие программного продукта.

3.3 Реализация фронтенда (React)

Клиентская часть веб-приложения разработана с использованием библиотеки React и реализована в виде одностраничного приложения (Single Page Application, SPA). Такой подход позволяет динамически обновлять интерфейс без полной перезагрузки страницы и обеспечивает высокую скорость взаимодействия пользователя с системой.

В качестве основных технологий фронтенд-разработки использовались:

- React;
- React Router;
- Axios;

- Context API;
- JavaScript;
- HTML;
- CSS.

Применение компонентного подхода позволяет разделить пользовательский интерфейс на независимые и переиспользуемые элементы, что упрощает разработку и сопровождение программного обеспечения.

3.3.1 Структура проекта React

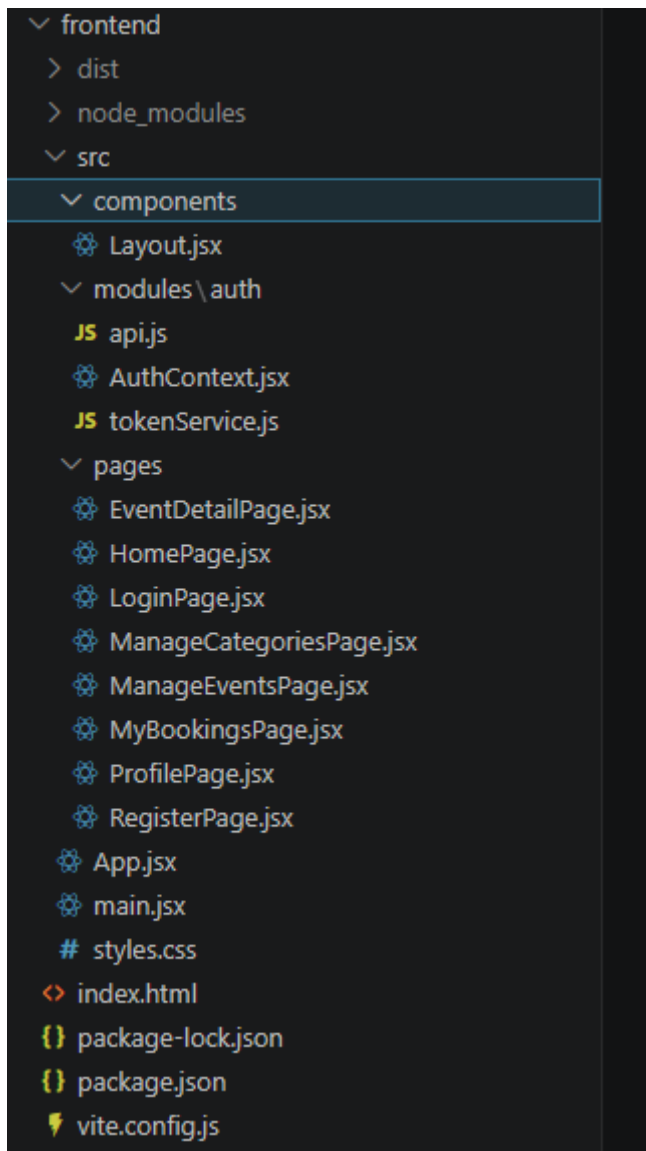
Проект клиентской части имеет модульную структуру и состоит из нескольких каталогов, каждый из которых отвечает за определённую функциональность.

Назначение основных каталогов:

- `components` — переиспользуемые компоненты интерфейса;
- `pages` — страницы приложения;
- `context` — управление глобальным состоянием;
- `services` — взаимодействие с REST API;
- `App.jsx` — основной компонент приложения;
- `main.jsx` — точка входа приложения.

Использование модульной структуры позволяет упростить сопровождение и дальнейшее развитие системы.

Рисунок 11 – Структура проекта React



3.3.2 Компоненты (ArticleList, ArticleDetail, Login, Register)

В соответствии с требованиями методических указаний в проекте используются компоненты, аналогичные ArticleList и ArticleDetail, адаптированные под предметную область бронирования билетов.

Компонент ArticleList выполняет функцию отображения списка мероприятий. Пользователь может просматривать доступные события и переходить к их подробному описанию.

Компонент ArticleDetail отображает детальную информацию о выбранном мероприятии:

- название;
- описание;

- дату проведения;
- место проведения;
- стоимость билета;
- комментарии пользователей.

Компонент Login предназначен для авторизации пользователя в системе.

После успешного входа пользователь получает JWT-токены доступа.

Компонент Register используется для регистрации новых пользователей.

Кроме перечисленных компонентов в приложении реализованы:

- страница профиля пользователя;
- страница бронирований;
- панель управления мероприятиями;
- управление категориями.

3.3.3 Маршрутизация (React Router)

Для организации навигации между страницами используется библиотека React Router.

Маршрутизация позволяет отображать различные компоненты в зависимости от адреса страницы без полной перезагрузки приложения.

В системе реализованы следующие маршруты:

Маршрут	Назначение
/	Главная страница
/login	Авторизация
/register	Регистрация
/events/:id	Просмотр мероприятия
/bookings	История бронирований
/profile	Личный кабинет

Для ограничения доступа к защищённым страницам используются специальные компоненты маршрутизации, обеспечивающие проверку

состояния авторизации пользователя.

Использование React Router позволяет повысить удобство работы с приложением и улучшить пользовательский опыт.

3.3.4 Управление состоянием (AuthContext)

Для хранения информации о пользователе и состоянии авторизации используется механизм React Context API.

Компонент AuthContext обеспечивает:

- хранение информации о пользователе;
- хранение JWT-токенов;
- выполнение входа в систему;
- выполнение выхода из системы;
- проверку статуса авторизации.

Использование Context API позволяет избежать передачи данных через множество вложенных компонентов и обеспечивает централизованное управление состоянием приложения.

После успешной авторизации данные пользователя сохраняются в глобальном состоянии приложения и становятся доступными для всех компонентов системы.

3.3.5 Взаимодействие с API (Axios, interceptors)

Для взаимодействия с серверной частью используется библиотека Axios.

Axios обеспечивает выполнение HTTP-запросов и обработку ответов сервера.

Основные возможности Axios:

- отправка GET-запросов;
- отправка POST-запросов;
- обработка ошибок;
- добавление заголовков авторизации.

Для доступа к защищённым ресурсам access token передаётся в HTTP-заголовке:

Authorization: Bearer <access_token>

В проекте реализован механизм Axios Interceptor, предназначенный для автоматического обновления JWT-токенов.

Алгоритм работы интерсептора:

1. Клиент отправляет запрос к API.
2. Сервер проверяет access token.
3. При получении ошибки 401 Unauthorized выполняется запрос обновления токена.
4. Сервер выдаёт новый access token.
5. Исходный запрос повторяется автоматически.

Использование интерсепторов повышает удобство работы пользователей и обеспечивает непрерывность пользовательской сессии.

Таким образом, клиентская часть приложения обеспечивает удобный пользовательский интерфейс и эффективное взаимодействие с серверной частью системы.

3.4 Рефакторинг и оптимизация

В процессе разработки программного обеспечения особое внимание уделялось качеству исходного кода, его читаемости и производительности. Для повышения эффективности работы приложения применялись методы оптимизации программного кода и организации взаимодействия с базой данных.

Использование современных средств разработки позволило обеспечить удобство сопровождения программного продукта и возможность его дальнейшего развития.

3.4.1 Статический анализ кода (flake8, ESLint)

Для повышения качества программного кода применялись общепринятые стандарты программирования и рекомендации по оформлению исходных текстов программ.

При разработке серверной части соблюдался стандарт оформления кода PEP 8, используемый в языке Python. Это позволило обеспечить единообразие структуры программного кода и повысить его читаемость.

При разработке клиентской части использовались рекомендации по написанию JavaScript-кода и организации React-компонентов.

Средства статического анализа позволяют:

- обнаруживать потенциальные ошибки;
- выявлять неиспользуемые переменные;
- контролировать стиль программирования;
- повышать качество исходного кода.

Для анализа Python-кода может использоваться инструмент flake8, а для анализа JavaScript-кода — ESLint.

Использование статического анализа способствует повышению надёжности программного обеспечения и облегчает его дальнейшее сопровождение.

3.4.2 Оптимизация запросов (select_related, prefetch_related)

Производительность веб-приложения во многом зависит от эффективности работы с базой данных.

Для оптимизации запросов Django ORM предоставляет механизмы `select_related()` и `prefetch_related()`, позволяющие уменьшить количество обращений к базе данных.

Метод `select_related()` используется для предварительной загрузки связанных объектов, связанных отношением «один к одному» или «многие к одному».

Пример использования:

```
events = Event.objects.select_related(
```

```
"category",  
"author"  
)
```

Метод `prefetch_related()` применяется для предварительной загрузки связанных коллекций объектов.

Пример использования:

```
events = Event.objects.prefetch_related(  
    "comments"  
)
```

Использование данных механизмов позволяет сократить количество SQL-запросов и повысить производительность приложения.

Даже если данные методы не используются во всех частях проекта, архитектура Django ORM позволяет внедрить их при дальнейшем развитии системы.

3.4.3 Кэширование на клиенте (React Query / RTK Query)

Для повышения производительности клиентских приложений широко используются механизмы кэширования данных.

Библиотеки `React Query` и `RTK Query` позволяют сохранять результаты запросов и уменьшать количество обращений к серверу.

В рамках данного проекта специализированные библиотеки клиентского кэширования не применялись. Повторное получение данных оптимизируется средствами `React`, браузерным кэшированием и повторным использованием состояния приложения.

Использование JWT-аутентификации и хранения данных пользователя в `Context API` позволяет сократить количество запросов к серверу и повысить отзывчивость интерфейса.

При дальнейшем развитии программного продукта возможно внедрение `React Query` или `RTK Query` для дополнительной оптимизации обмена данными между клиентом и сервером.

Таким образом, применённые методы рефакторинга и оптимизации позволяют обеспечить высокую производительность и удобство сопровождения разработанного программного обеспечения.

3.5 Безопасность и транзакции

Одной из важнейших задач при разработке веб-приложений является обеспечение безопасности пользовательских данных и защита системы от несанкционированного доступа.

В разработанном приложении реализованы механизмы аутентификации, разграничения прав доступа и проверки корректности вводимых данных.

3.5.1 JWT-аутентификация

Для аутентификации пользователей используется технология JSON Web Token (JWT).

После успешного входа пользователя сервер генерирует два токена:

- access token — используется для доступа к защищённым ресурсам системы;
- refresh token — используется для обновления access token после истечения срока его действия.

Полученные токены сохраняются в localStorage браузера и автоматически используются при выполнении запросов к серверу.

Передача токена выполняется посредством HTTP-заголовка:

Authorization: Bearer <access_token>

При истечении срока действия access token клиентское приложение автоматически получает новый токен с использованием refresh token, что обеспечивает непрерывность пользовательской сессии.

Использование JWT-аутентификации позволяет отказаться от хранения серверных сессий и повысить масштабируемость приложения.

3.5.2 Защита от XSS, CSRF (CORS настройки)

Для защиты веб-приложения от распространённых угроз информационной безопасности были реализованы соответствующие механизмы защиты.

Защита от межсайтового выполнения сценариев (XSS) обеспечивается:

- экранированием данных в React;
- проверкой пользовательского ввода;
- ограничением выполнения произвольного JavaScript-кода.

Защита от подделки межсайтовых запросов (CSRF) обеспечивается использованием JWT-аутентификации вместо традиционной cookie-аутентификации.

Для взаимодействия клиентской и серверной частей приложения были настроены правила CORS.

Пример настройки:

`CORS_ALLOW_ALL_ORIGINS = True`

Использование CORS позволяет безопасно выполнять запросы между React-приложением и сервером Django.

3.5.3 Валидация данных на сервере и клиенте

Проверка корректности вводимых данных является важным механизмом обеспечения надёжности программного обеспечения.

На клиентской стороне выполняется:

- проверка обязательных полей;
- контроль формата вводимых данных;
- отображение сообщений об ошибках пользователю.

На серверной стороне валидация реализована средствами Django REST Framework и сериализаторов.

Сериализаторы обеспечивают:

- проверку типов данных;
- контроль обязательных полей;

- предотвращение сохранения некорректных данных.

Использование многоуровневой валидации повышает безопасность приложения и обеспечивает целостность данных.

Таким образом, реализованные механизмы безопасности позволяют обеспечить надёжную защиту данных пользователей и корректную работу системы.

5. РАЗВЁРТЫВАНИЕ И ЭКСПЛУАТАЦИЯ

5.1. Контейнеризация (Docker)

В рамках курсового проекта контейнеризация приложения не выполнялась.

5.2 Инструкция по установке

Для установки и запуска разработанного веб-приложения необходимо предварительно установить программное обеспечение, используемое в проекте.

Перед началом работы необходимо убедиться в наличии следующих компонентов:

- Python версии 3.10 и выше;
- Node.js версии 18 и выше;
- пакетного менеджера npm;
- системы контроля версий Git (при необходимости).

Процесс установки приложения состоит из нескольких этапов.

Установка серверной части

Сначала необходимо получить исходный код проекта и перейти в каталог серверной части приложения:

```
git clone <repository_url>
```

```
cd backend
```

Далее создаётся виртуальное окружение Python:

```
python -m venv venv
```

Активация виртуального окружения в операционной системе Windows выполняется командой:

```
venv\Scripts\activate
```

После активации виртуального окружения устанавливаются необходимые зависимости:

```
pip install -r requirements.txt
```

После установки зависимостей выполняются миграции базы данных:

```
python manage.py makemigrations
```

```
python manage.py migrate
```

При необходимости создаётся учётная запись администратора:

```
python manage.py createsuperuser
```

Запуск сервера разработки выполняется командой:

```
python manage.py runserver
```

После запуска сервер будет доступен по адресу:

```
http://127.0.0.1:8000/
```

Установка клиентской части

Для установки клиентской части необходимо перейти в каталог React-приложения:

```
cd frontend
```

Установка зависимостей выполняется командой:

```
npm install
```

Запуск клиента выполняется командой:

```
npm run dev
```

После запуска приложение будет доступно по адресу:

```
http://localhost:5173/
```

В результате выполнения указанных действий система полностью готова к эксплуатации и взаимодействию пользователей.

5.3 Инструкция по настройке (.env, CORS)

После установки программного обеспечения необходимо выполнить настройку параметров приложения.

Настройка конфигурационных параметров

Для хранения конфиденциальных данных и параметров окружения могут

использоваться переменные среды, размещённые в файле .env.

Пример содержимого файла .env:

```
SECRET_KEY=your_secret_key
```

```
DEBUG=True
```

```
ALLOWED_HOSTS=localhost,127.0.0.1
```

Использование переменных окружения позволяет повысить безопасность приложения и упростить перенос системы между различными средами выполнения.

Настройка CORS

Поскольку клиентская и серверная части приложения работают на разных портах, необходимо разрешить междоменные запросы.

Для этого используется библиотека `django-cors-headers`.

В файл `settings.py` добавляются следующие настройки:

```
INSTALLED_APPS = [
```

```
...
```

```
"corsheaders",
```

```
]
```

Подключение middleware:

```
MIDDLEWARE = [
```

```
"corsheaders.middleware.CorsMiddleware",
```

```
...
```

```
]
```

Разрешение запросов:

```
CORS_ALLOW_ALL_ORIGINS = True
```

В рабочей среде рекомендуется указывать только доверенные адреса:

```
CORS_ALLOWED_ORIGINS = [
```

```
"http://localhost:5173"
```

```
]
```

Настройка CORS обеспечивает корректное взаимодействие React-приложения и Django REST API.

5.4 Требования к окружению (Python, Node.js, PostgreSQL)

Для корректной работы разработанного программного обеспечения требуется следующее программное окружение.

Таблица 11 - Требования к программному окружению

Компонент	Минимальная версия
Python	3.10
Django	5.x
Django REST Framework	3.x
Node.js	18.x
npm	9.x
React	18.x
SQLite	3.x

В ходе разработки использовалась встроенная СУБД SQLite, которая не требует отдельной установки и настройки.

При необходимости дальнейшего развития проекта возможен переход на СУБД PostgreSQL, обеспечивающую более высокую производительность и масштабируемость системы.

Основные аппаратные требования:

- процессор с тактовой частотой не менее 2 ГГц;
- оперативная память не менее 4 ГБ;
- свободное место на диске не менее 1 ГБ;
- наличие современного веб-браузера.

Разработанное приложение является кроссплатформенным и может функционировать под управлением операционных систем Windows, Linux и macOS.

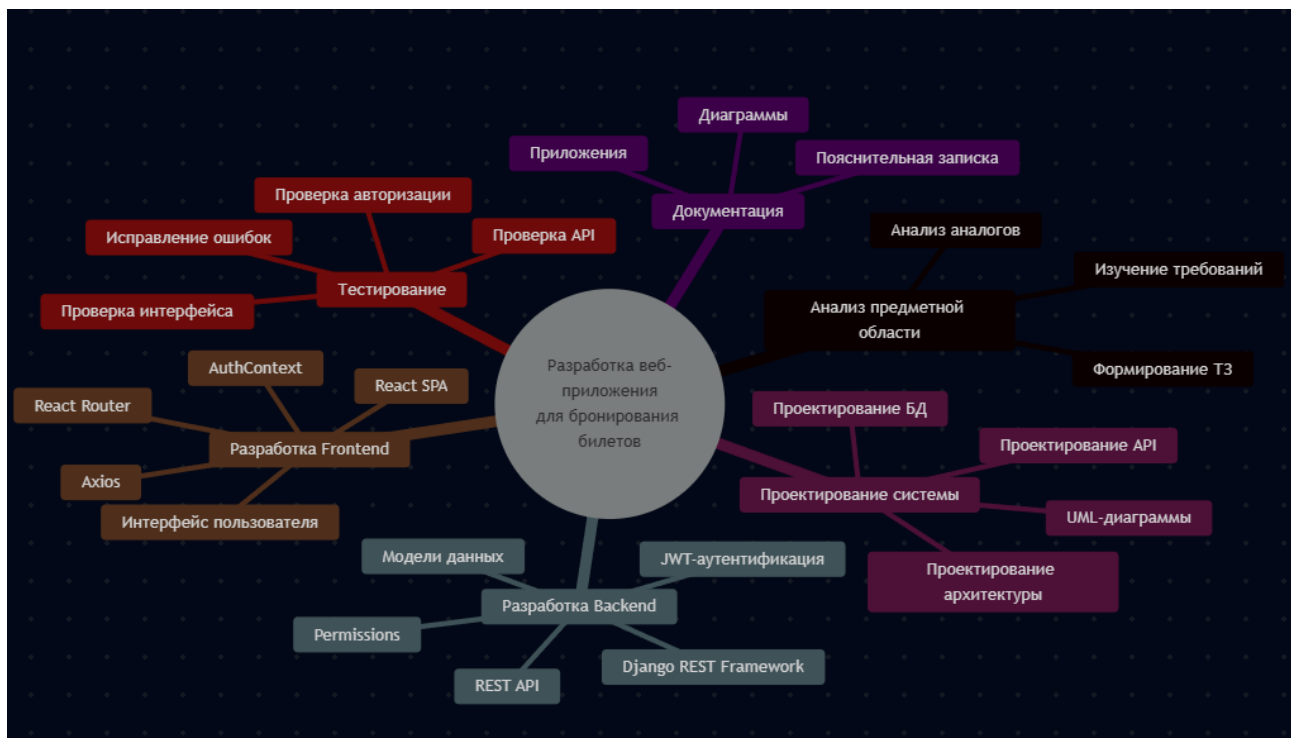
6. УПРАВЛЕНИЕ ПРОЕКТОМ

Управление проектом является важной частью процесса разработки программного обеспечения и позволяет организовать выполнение работ, оценить трудоёмкость проекта и выявить возможные риски.

При разработке веб-приложения для бронирования билетов применялся поэтапный подход, включающий анализ требований, проектирование, реализацию и тестирование программного продукта.

6.1 WBS (иерархическая структура работ)

Рисунок 12 - Иерархическая структура работ (WBS)



6.2 Диаграмма Ганта

Рисунок 13 - План выполнения работ



6.3 Оценка трудозатрат (COCOMO)

Для оценки трудоёмкости разработки программного обеспечения использовалась модель COCOMO (Constructive Cost Model).

Разрабатываемое веб-приложение относится к категории Organic, поскольку проект обладает следующими характеристиками:

- небольшой размер команды разработчиков;
- использование стандартных технологий;
- средняя сложность программного продукта;
- ограниченная предметная область.

Базовая модель COCOMO рассчитывается по формуле:

$$[E = 2.4 \times (KLOC)^{1.05}]$$

где:

- E — трудозатраты в человеко-месяцах;
- KLOC — количество тысяч строк исходного кода.

Предположим, что общий объём проекта составляет около 8 KLOC (около 8000 строк кода).

Тогда:

$$[E = 2.4 \times 8^{1.05} \approx 21.3]$$

Полученное значение показывает, что трудоёмкость проекта составляет приблизительно 21 человеко-месяц.

При выполнении проекта одним разработчиком фактическое время реализации составило около одного месяца, что соответствует учебному характеру разработки.

Использование модели COCOMO позволяет предварительно оценивать ресурсы и сроки выполнения программных проектов.

6.4 Управление рисками

В процессе разработки программного обеспечения могут возникать различные риски, способные повлиять на сроки выполнения проекта и качество итогового продукта.

Основные риски проекта представлены в таблице.

Таблица 12 - Анализ рисков проекта

Риск	Вероятность	Последствия	Способ минимизации
Ошибки в программном коде	Средняя	Некорректная работа системы	Тестирование и отладка
Потеря данных	Низкая	Потеря информации	Резервное копирование
Ошибки интеграции API	Средняя	Нарушение обмена данными	Проверка API
Нарушение безопасности	Низкая	Несанкционированный доступ	JWT и permissions
Изменение требований	Средняя	Увеличение сроков	Гибкая архитектура
Ошибки пользователя	Высокая	Некорректный ввод данных	Валидация данных

Проведённый анализ показал, что большинство рисков могут быть успешно снижены посредством применения современных технологий разработки, тестирования и обеспечения безопасности.

ЗАКЛЮЧЕНИЕ

Выводы

В ходе выполнения курсовой работы была разработана информационная система для бронирования билетов, реализованная в виде современного веб-приложения на основе клиент-серверной архитектуры.

В процессе выполнения работы был проведён анализ предметной области, исследованы существующие решения и определены требования к

разрабатываемому программному обеспечению. Выполнено проектирование архитектуры системы, базы данных и программных компонентов приложения.

Серверная часть приложения реализована с использованием фреймворка Django REST Framework, обеспечивающего создание REST API и обработку запросов пользователей. В качестве системы управления базами данных использовалась SQLite. Для обеспечения безопасности доступа к ресурсам реализован механизм JWT-аутентификации на основе access и refresh токенов.

Клиентская часть разработана с использованием библиотеки React и реализована в виде одностраничного приложения (SPA). Для организации маршрутизации использовался React Router, а взаимодействие с сервером осуществлялось посредством библиотеки Axios. Реализован механизм автоматического обновления токенов доступа с использованием Axios Interceptor.

В результате выполнения работы были закреплены практические навыки проектирования и разработки современных веб-приложений, а также освоены технологии Django REST Framework и React.

Результаты

В ходе выполнения курсового проекта были достигнуты следующие результаты:

- разработана клиент-серверная архитектура приложения;
- реализована серверная часть на основе Django REST Framework;
- создан REST API для взаимодействия компонентов системы;
- реализована JWT-аутентификация пользователей;
- разработан пользовательский интерфейс на React;
- реализовано разграничение прав доступа между гостями, пользователями и администраторами;
- создана база данных для хранения информации о мероприятиях и бронированиях;
- реализованы CRUD-операции для работы с данными;
- выполнено тестирование основных функций приложения;
- подготовлена проектная и техническая документация.

Таким образом, поставленная цель курсовой работы — разработка веб-приложения для бронирования билетов — была достигнута в полном объёме, а все поставленные задачи успешно решены.

Перспективы развития

Разработанное программное обеспечение обладает возможностью дальнейшего развития и расширения функциональности.

В качестве основных направлений модернизации системы можно выделить:

- интеграцию платёжных систем для онлайн-оплаты билетов;
- реализацию уведомлений по электронной почте;
- внедрение системы рейтингов и отзывов пользователей;
- разработку мобильного приложения;
- интеграцию с внешними сервисами продажи билетов;
- внедрение системы аналитики и статистики посещаемости мероприятий;
- переход на промышленную СУБД PostgreSQL;
- развёртывание приложения в облачной инфраструктуре.

Дальнейшее развитие системы позволит повысить её функциональные возможности, производительность и удобство использования для конечных пользователей.

В результате выполненной работы разработано полнофункциональное веб-приложение для бронирования билетов, соответствующее современным требованиям к разработке информационных систем и обладающее потенциалом для дальнейшего совершенствования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Кузьмин, А. Г. Разработка фронтенд-приложений / А. Г. Кузьмин. — Санкт-Петербург : Питер, 2025. — 496 с. — (Библиотека программиста). — ISBN 978-5-4461-4272-9.
2. Кухтин, С. А. Frontend-разработка сайтов и приложений на HTML, CSS, JavaScript и React / С. А. Кухтин. — Санкт-Петербург : Наука и Техника, 2026. — 512 с. — ISBN 978-5-907592-84-1.
3. Орбайсета, А. С. Создание фронтенд-фреймворка с нуля / А. С. Орбайсета. — Санкт-Петербург : Питер, 2025. — 384 с. — (Библиотека программиста). — ISBN 978-5-4461-4201-9.
4. Курс «Фронтенд-разработчик» от Яндекс Практикум. — URL: <https://practicum.yandex.ru/frontend-developer/>.
5. Проектирование и разработка WEB-приложений. Введение в frontend и backend разработку на JavaScript и node.js : учебное пособие для вузов. — 4-е изд., стер. — Санкт-Петербург : Лань, 2025. — 120 с. — ISBN 978-5-507-51011-5
6. Чернышев, С. А. Основы Flutter / С. А. Чернышев, Ю. М. Петров, С. П. Ильин,
7. Григорьев В. В. Python и Django. Разработка веб-приложений. — СПб.: Питер, 2024. — 512 с.

8. Дронов В. А. Django 5. Практика создания веб-сайтов. – СПб.: БХВ-Петербург, 2024. – 544 с.
9. Ломов А. Ю. Веб-разработка на Django 5. – М.: ДМК Пресс, 2025. – 412 с.
10. Документация Django. URL: <https://docs.djangoproject.com/>
11. Документация Django REST Framework. URL: <https://www.django-rest-framework.org/>
12. Django REST Framework Simple JWT Documentation. URL: <https://django-rest-framework-simplejwt.readthedocs.io/>
13. Документация React. URL: <https://react.dev/>
14. Документация React Router. URL: <https://reactrouter.com/>
15. Документация Axios. URL: <https://axios-http.com/>
16. Документация SQLite. URL: <https://sqlite.org/>
17. Документация JWT. URL: <https://jwt.io/>
18. Документация Visual Studio Code. URL: <https://code.visualstudio.com/>
19. Документация Git. URL: <https://git-scm.com/>

Приложения

Приложение А. Листинги кода (models.py, serializers.py, consumers.py, React components)

А.1 models.py

backend/tickets/models.py

```
from decimal import Decimal
from uuid import uuid4

from django.conf import settings
from django.db import models
from django.utils.text import slugify

class EventCategory(models.Model):
    name = models.CharField(max_length=100, unique=True)
    description = models.TextField()
    icon = models.CharField(max_length=50, blank=True)
    color = models.CharField(max_length=20, default='#1d4ed8')
    age_limit = models.PositiveIntegerField(default=0)

    class Meta:
        ordering = ['name']
        verbose_name = 'категория событий'
        verbose_name_plural = 'Категории событий'

    def __str__(self):
        return self.name
```

```

class Event(models.Model):
    STATUS_CHOICES = (
        ('draft', 'Черновик'),
        ('published', 'Опубликовано'),
        ('sold_out', 'Нет мест'),
    )

    category = models.ForeignKey(EventCategory, on_delete=models.PROTECT,
related_name='events')
    author = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE,
related_name='events')
    title = models.CharField(max_length=200)
    slug = models.SlugField(max_length=220, unique=True, blank=True)
    description = models.TextField()
    event_date = models.DateTimeField()
    city = models.CharField(max_length=120)
    venue_name = models.CharField(max_length=160)
    venue_address = models.CharField(max_length=255)
    organizer_name = models.CharField(max_length=160)
    total_tickets = models.PositiveIntegerField(default=100)
    available_tickets = models.PositiveIntegerField(default=100)
    price = models.DecimalField(max_digits=10, decimal_places=2,
default=Decimal('1500.00'))
    image_url = models.URLField(blank=True)
    status = models.CharField(max_length=20, choices=STATUS_CHOICES,
default='draft')
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ['event_date', '-created_at']
        verbose_name = 'событие'
        verbose_name_plural = 'События'

    def save(self, *args, **kwargs):
        if not self.slug:
            base_slug = slugify(self.title) or f'event-{uuid4().hex[:8]}'
            slug = base_slug
            suffix = 1
            while Event.objects.exclude(pk=self.pk).filter(slug=slug).exists():
                slug = f'{base_slug}-{suffix}'
                suffix += 1
            self.slug = slug
        if self.available_tickets == 0:
            self.status = 'sold_out'
        super().save(*args, **kwargs)

    def __str__(self):
        return self.title

```

```

class Booking(models.Model):
    STATUS_CHOICES = (
        ('active', 'Активно'),
        ('cancelled', 'Отменено'),
    )

    event = models.ForeignKey(Event, on_delete=models.CASCADE,
related_name='bookings')
    user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE,
related_name='bookings')
    ticket_quantity = models.PositiveIntegerField(default=1)
    total_price = models.DecimalField(max_digits=10, decimal_places=2)
    contact_phone = models.CharField(max_length=20)
    attendee_name = models.CharField(max_length=160)
    status = models.CharField(max_length=20, choices=STATUS_CHOICES,
default='active')
    booked_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['-booked_at']
        verbose_name = 'бронирование'
        verbose_name_plural = 'Бронирования'

    def __str__(self):
        return f'{self.user} -> {self.event}'

class Comment(models.Model):
    event = models.ForeignKey(Event, on_delete=models.CASCADE,
related_name='comments')
    author = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE,
related_name='comments')
    text = models.TextField()
    rating = models.PositiveIntegerField(default=5)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ['-created_at']
        verbose_name = 'комментарий'
        verbose_name_plural = 'Комментарии'

    def __str__(self):
        return f'Комментарий {self.author} к {self.event}'

```

backend/users/models.py

```

from django.contrib.auth.models import AbstractUser

```

```

from django.db import models

class User(AbstractUser):
    phone = models.CharField(max_length=20, blank=True)
    bio = models.TextField(blank=True)
    avatar_url = models.URLField(blank=True)

    class Meta:
        verbose_name = 'пользователь'
        verbose_name_plural = 'Пользователи'

    def __str__(self):
        return self.username

class Profile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE,
related_name='profile')
    city = models.CharField(max_length=120, blank=True)
    preferred_event_type = models.CharField(max_length=120, blank=True)
    notification_email = models.EmailField(blank=True)
    about = models.TextField(blank=True)

    class Meta:
        verbose_name = 'профиль'
        verbose_name_plural = 'Профили'

    def __str__(self):
        return f'Профиль: {self.user.username}'

```

A.2 serializers.py

backend/tickets/serializers.py

```

from decimal import Decimal

from rest_framework import serializers

from users.serializers import UserSerializer

from .models import Booking, Comment, Event, EventCategory

class EventCategorySerializer(serializers.ModelSerializer):
    class Meta:
        model = EventCategory

```

```

        fields = '__all__'

class EventListSerializer(serializers.ModelSerializer):
    category = EventCategorySerializer(read_only=True)
    author_name = serializers.CharField(source='author.username', read_only=True)
    author_id = serializers.IntegerField(source='author.id', read_only=True)

    class Meta:
        model = Event
        fields = (
            'id',
            'category',
            'author_id',
            'author_name',
            'title',
            'slug',
            'description',
            'event_date',
            'city',
            'venue_name',
            'venue_address',
            'organizer_name',
            'total_tickets',
            'available_tickets',
            'price',
            'image_url',
            'status',
            'created_at',
        )

class EventWriteSerializer(serializers.ModelSerializer):
    class Meta:
        model = Event
        fields = (
            'id',
            'category',
            'title',
            'description',
            'event_date',
            'city',
            'venue_name',
            'venue_address',
            'organizer_name',
            'total_tickets',
            'available_tickets',
            'price',
            'image_url',
            'status',
        )

```

```

class CommentSerializer(serializers.ModelSerializer):
    author_name = serializers.CharField(source='author.username', read_only=True)

    class Meta:
        model = Comment
        fields = ('id', 'event', 'author', 'author_name', 'text', 'rating',
'created_at', 'updated_at')
        read_only_fields = ('author',)

class BookingSerializer(serializers.ModelSerializer):
    user = UserSerializer(read_only=True)
    event_title = serializers.CharField(source='event.title', read_only=True)

    class Meta:
        model = Booking
        fields = (
            'id',
            'event',
            'event_title',
            'user',
            'ticket_quantity',
            'total_price',
            'contact_phone',
            'attendee_name',
            'status',
            'booked_at',
        )
        read_only_fields = ('total_price', 'status')

    def validate(self, attrs):
        event = attrs['event']
        quantity = attrs['ticket_quantity']
        if quantity <= 0:
            raise serializers.ValidationError('Количество билетов должно быть
положительным.')
        if event.available_tickets < quantity:
            raise serializers.ValidationError('Недостаточно свободных билетов.')
        return attrs

    def create(self, validated_data):
        event = validated_data['event']
        quantity = validated_data['ticket_quantity']
        validated_data['total_price'] = Decimal(quantity) * event.price
        event.available_tickets -= quantity
        if event.available_tickets == 0:
            event.status = 'sold_out'
        event.save()
        return super().create(validated_data)

```

```

def update(self, instance, validated_data):
    new_status = validated_data.get('status', instance.status)
    if instance.status == 'active' and new_status == 'cancelled':
        instance.event.available_tickets += instance.ticket_quantity
        if instance.event.status == 'sold_out':
            instance.event.status = 'published'
        instance.event.save()
    return super().update(instance, validated_data)

class EventDetailSerializer(EventListSerializer):
    comments = CommentSerializer(many=True, read_only=True)

    class Meta(EventListSerializer.Meta):
        fields = EventListSerializer.Meta.fields + ('total_tickets', 'updated_at',
        'comments')

```

backend/users/serializers.py

```

from django.contrib.auth.password_validation import validate_password
from rest_framework import serializers

from .models import Profile, User

class RegisterSerializer(serializers.ModelSerializer):
    password = serializers.CharField(write_only=True,
    validators=[validate_password])

    class Meta:
        model = User
        fields = (
            'id',
            'username',
            'email',
            'password',
            'first_name',
            'last_name',
            'phone',
            'bio',
            'avatar_url',
        )

    def create(self, validated_data):
        password = validated_data.pop('password')
        user = User(**validated_data)
        user.set_password(password)
        user.save()
        Profile.objects.create(user=user, notification_email=user.email)

```



```

        return user

class ProfileSerializer(serializers.ModelSerializer):
    class Meta:
        model = Profile
        fields = ('city', 'preferred_event_type', 'notification_email', 'about')

class UserSerializer(serializers.ModelSerializer):
    profile = ProfileSerializer()

    class Meta:
        model = User
        fields = (
            'id',
            'username',
            'email',
            'first_name',
            'last_name',
            'phone',
            'bio',
            'avatar_url',
            'is_staff',
            'profile',
        )
        read_only_fields = ('is_staff',)

    def update(self, instance, validated_data):
        profile_data = validated_data.pop('profile', {})
        for attr, value in validated_data.items():
            setattr(instance, attr, value)
        instance.save()

        profile, _ = Profile.objects.get_or_create(user=instance)
        for attr, value in profile_data.items():
            setattr(profile, attr, value)
        profile.save()
        return instance

```

A.3 consumers.py

В рамках данного проекта технология WebSocket не использовалась, поэтому файл consumers.py отсутствует.

A.4 React components

frontend/src/pages/Login.jsx

```
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';

import { useAuth } from '../modules/auth/AuthContext';

export function LoginPage() {
  const navigate = useNavigate();
  const { login } = useAuth();
  const [form, setForm] = useState({ username: '', password: '' });

  async function handleSubmit(event) {
    event.preventDefault();
    await login(form);
    navigate('/');
  }

  return (
    <section className="form-shell">
      <form className="panel form-card stack" onSubmit={handleSubmit}>
        <h2>Вход</h2>
        <input
          className="input"
          placeholder="Логин"
          value={form.username}
          onChange={(event) => setForm((prev) => ({ ...prev, username:
event.target.value })))}
        />
        <input
          className="input"
          placeholder="Пароль"
          type="password"
          value={form.password}
          onChange={(event) => setForm((prev) => ({ ...prev, password:
event.target.value })))}
        />
        <button className="primary-button" type="submit">
          Войти
        </button>
      </form>
    </section>
  );
}
```

frontend/src/pages/Register.jsx

```

import { useState } from 'react';
import { useNavigate } from 'react-router-dom';

import { useAuth } from '../modules/auth/AuthContext';

const initialForm = {
  username: '',
  email: '',
  password: '',
  first_name: '',
  last_name: '',
  phone: '',
};

export function RegisterPage() {
  const navigate = useNavigate();
  const { register } = useAuth();
  const [form, setForm] = useState(initialForm);
  const [error, setError] = useState('');
  const [submitting, setSubmitting] = useState(false);

  async function handleSubmit(event) {
    event.preventDefault();
    setError('');
    setSubmitting(true);

    try {
      await register(form);
      navigate('/');
    } catch (requestError) {
      const data = requestError?.response?.data;
      if (data && typeof data === 'object') {
        const messages = Object.entries(data)
          .flatMap(([field, value]) => {
            const items = Array.isArray(value) ? value : [value];
            return items.map((item) => `${field}: ${item}`);
          })
          .join(' ');
        setError(messages || 'Не удалось создать аккаунт.');
```

```

      } else {
        setError('Не удалось создать аккаунт. Проверьте введённые данные.');
```

```

      }
    } finally {
      setSubmitting(false);
    }
  }

  return (
    <section className="form-shell">
      <form className="panel form-card stack" onSubmit={handleSubmit}>
```

```

    <h2>Регистрация</h2>
    <p className="muted">Используйте пароль минимум из 8 символов, не только
цифры.</p>
    {Object.entries({
      username: 'Логин',
      email: 'Email',
      password: 'Пароль',
      first_name: 'Имя',
      last_name: 'Фамилия',
      phone: 'Телефон',
    }).map(([key, label]) => (
      <input
        key={key}
        className="input"
        placeholder={label}
        type={key === 'password' ? 'password' : 'text'}
        value={form[key]}
        onChange={(currentEvent) =>
          setForm((prev) => ({ ...prev, [key]: currentEvent.target.value }))
        }
      />
    ))}
    {error && <p className="form-error">{error}</p>}
    <button className="primary-button" disabled={submitting} type="submit">
      {submitting ? 'Создание...' : 'Создать аккаунт'}
    </button>
  </form>
</section>
);
}

```

frontend/src/pages/EventList.jsx

```

import { useEffect, useState } from 'react';

import { api } from '../modules/auth/api';
import { useAuth } from '../modules/auth/AuthContext';

const emptyForm = {
  category: '',
  title: '',
  description: '',
  event_date: '',
  city: '',
  venue_name: '',
  venue_address: '',
  organizer_name: '',
  total_tickets: 100,
  available_tickets: 100,

```

```

    price: 1500,
    image_url: '',
    status: 'draft',
  };

export function ManageEventsPage() {
  const { user } = useAuth();
  const [events, setEvents] = useState([]);
  const [categories, setCategories] = useState([]);
  const [form, setForm] = useState(emptyForm);
  const [editingId, setEditingId] = useState(null);

  async function loadData() {
    const [eventsResponse, categoriesResponse] = await Promise.all([
      api.get('/events/?mine=1'),
      api.get('/categories/'),
    ]);
    setEvents(eventsResponse.data.results ?? eventsResponse.data);
    setCategories(categoriesResponse.data.results ?? categoriesResponse.data);
  }

  useEffect(() => {
    loadData();
  }, []);

  async function handleSubmit(event) {
    event.preventDefault();
    const payload = {
      ...form,
      category: Number(form.category),
      total_tickets: Number(form.total_tickets),
      available_tickets: Number(form.available_tickets),
      price: Number(form.price),
    };

    if (editingId) {
      await api.put(`/events/${editingId}/`, payload);
    } else {
      await api.post('/events/', payload);
    }

    setForm(emptyForm);
    setEditingId(null);
    await loadData();
  }

  async function handleDelete(id) {
    await api.delete(`/events/${id}/`);
    await loadData();
  }
}

```

```

function startEdit(event) {
  setEditingId(event.id);
  setForm({
    category: event.category.id,
    title: event.title,
    description: event.description,
    event_date: event.event_date.slice(0, 16),
    city: event.city,
    venue_name: event.venue_name,
    venue_address: event.venue_address,
    organizer_name: event.organizer_name,
    total_tickets: event.total_tickets ?? event.available_tickets,
    available_tickets: event.available_tickets,
    price: event.price,
    image_url: event.image_url ?? '',
    status: event.status,
  });
}

return (
  <section className="detail-layout">
    <form className="panel stack" onSubmit={handleSubmit}>
      <h2>{editingId ? 'Редактирование события' : 'Новое событие'}</h2>
      <select
        className="input"
        value={form.category}
        onChange={(event) => setForm((prev) => ({ ...prev, category:
event.target.value })))}
      >
        <option value="">Выберите категорию</option>
        {categories.map((category) => (
          <option key={category.id} value={category.id}>
            {category.name}
          </option>
        ))}
      </select>
      {Object.entries({
        title: 'Название',
        city: 'Город',
        venue_name: 'Площадка',
        venue_address: 'Адрес',
        organizer_name: 'Организатор',
        event_date: 'Дата и время',
        total_tickets: 'Всего билетов',
        available_tickets: 'Доступно билетов',
        price: 'Цена',
        image_url: 'Ссылка на изображение',
      }).map(([key, label]) => (
        <input

```

```

        key={key}
        className="input"
        placeholder={label}
        type={key === 'event_date' ? 'datetime-local' : 'text'}
        value={form[key]}
        onChange={(event) => setForm((prev) => ({ ...prev, [key]:
event.target.value })))}
      />
    ))}
    <textarea
      className="input input--textarea"
      placeholder="Описание события"
      value={form.description}
      onChange={(event) => setForm((prev) => ({ ...prev, description:
event.target.value })))}
    />
    <select
      className="input"
      value={form.status}
      onChange={(event) => setForm((prev) => ({ ...prev, status:
event.target.value })))}
    >
      <option value="draft">Черновик</option>
      <option value="published">Опубликовано</option>
      <option value="sold_out">Нет мест</option>
    </select>
    <button className="primary-button" type="submit">
      {editingId ? 'Сохранить' : 'Создать'}
    </button>
  </form>

  <div className="stack">
    <h2>Мои события</h2>
    {events.map((event) => (
      <article className="panel booking-card" key={event.id}>
        <h3>{event.title}</h3>
        <p>
          {event.city}, {event.venue_name}
        </p>
        <p>Статус: {event.status}</p>
        {user?.is_staff && <p>Автор: {event.author_name}</p>}
        <div className="actions">
          <button className="ghost-button" onClick={() => startEdit(event)}
type="button">
            Редактировать
          </button>
          <button className="ghost-button" onClick={() =>
handleDelete(event.id)} type="button">
            Удалить
          </button>
        </div>
      </article>
    ))}
  </div>

```

```

        </div>
      </article>
    )})
  </div>
</section>
);
}

```

frontend/src/modules/api.js

```

import axios from 'axios';

import { clearTokens, getAccessToken, getRefreshToken, setTokens } from
'./tokenService';

const API_URL = import.meta.env.VITE_API_URL ?? 'http://127.0.0.1:8000/api';
const AUTH_URL = 'http://127.0.0.1:8000/api/auth';

export const api = axios.create({
  baseURL: API_URL,
});

api.interceptors.request.use((config) => {
  const token = getAccessToken();
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

api.interceptors.response.use(
  (response) => response,
  async (error) => {
    const originalRequest = error.config;
    if (
      error.response?.status === 401 &&
      !originalRequest._retry &&
      getRefreshToken()
    ) {
      originalRequest._retry = true;
      try {
        const refreshResponse = await axios.post(`${AUTH_URL}/token/refresh/`, {
          refresh: getRefreshToken(),
        });
        setTokens(refreshResponse.data.access, refreshResponse.data.refresh ??
getRefreshToken());
        originalRequest.headers.Authorization = `Bearer
${refreshResponse.data.access}`;
        return api(originalRequest);
      } catch (err) {
        // If refresh token is also invalid, clear tokens and redirect to login
        clearTokens();
        // ...
      }
    }
    return Promise.reject(error);
  }
);

```



```

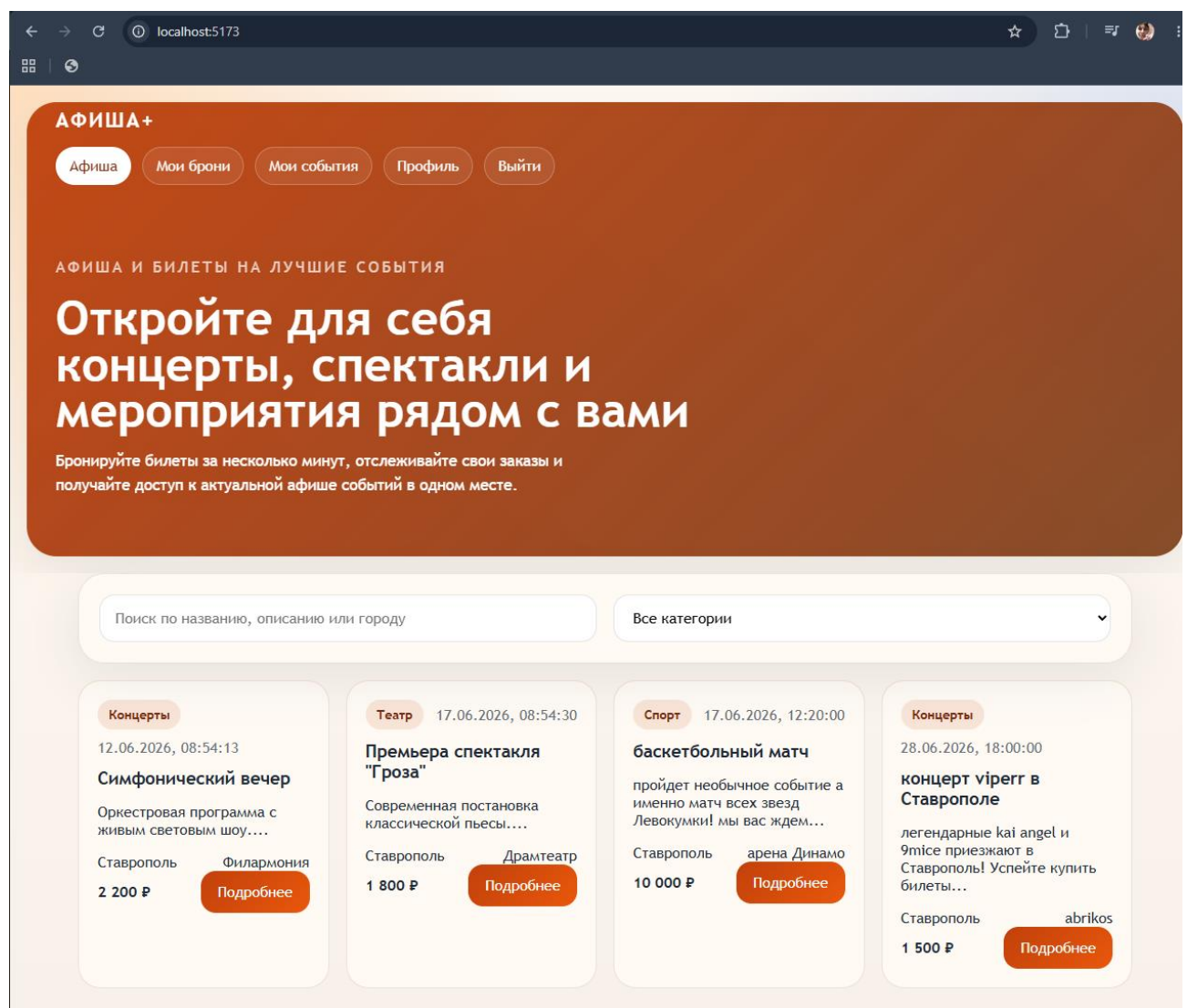
    } catch (refreshError) {
      clearTokens();
      return Promise.reject(refreshError);
    }
  }
  return Promise.reject(error);
},
);

export const authApi = axios.create({
  baseURL: AUTH_URL,
});

```

Приложение Б. Скриншоты интерфейсов

Б.1 Главная страница приложения



Б.2 Страница регистрации

АФИША+

Афиша Вход Регистрация

АФИША И БИЛЕТЫ НА ЛУЧШИЕ СОБЫТИЯ

Откройте для себя концерты, спектакли и мероприятия рядом с вами

Бронируйте билеты за несколько минут, отслеживайте свои заказы и получайте доступ к актуальной афише событий в одном месте.

Регистрация

Используйте пароль минимум из 8 символов, не только цифры.

Логин

Email

Пароль

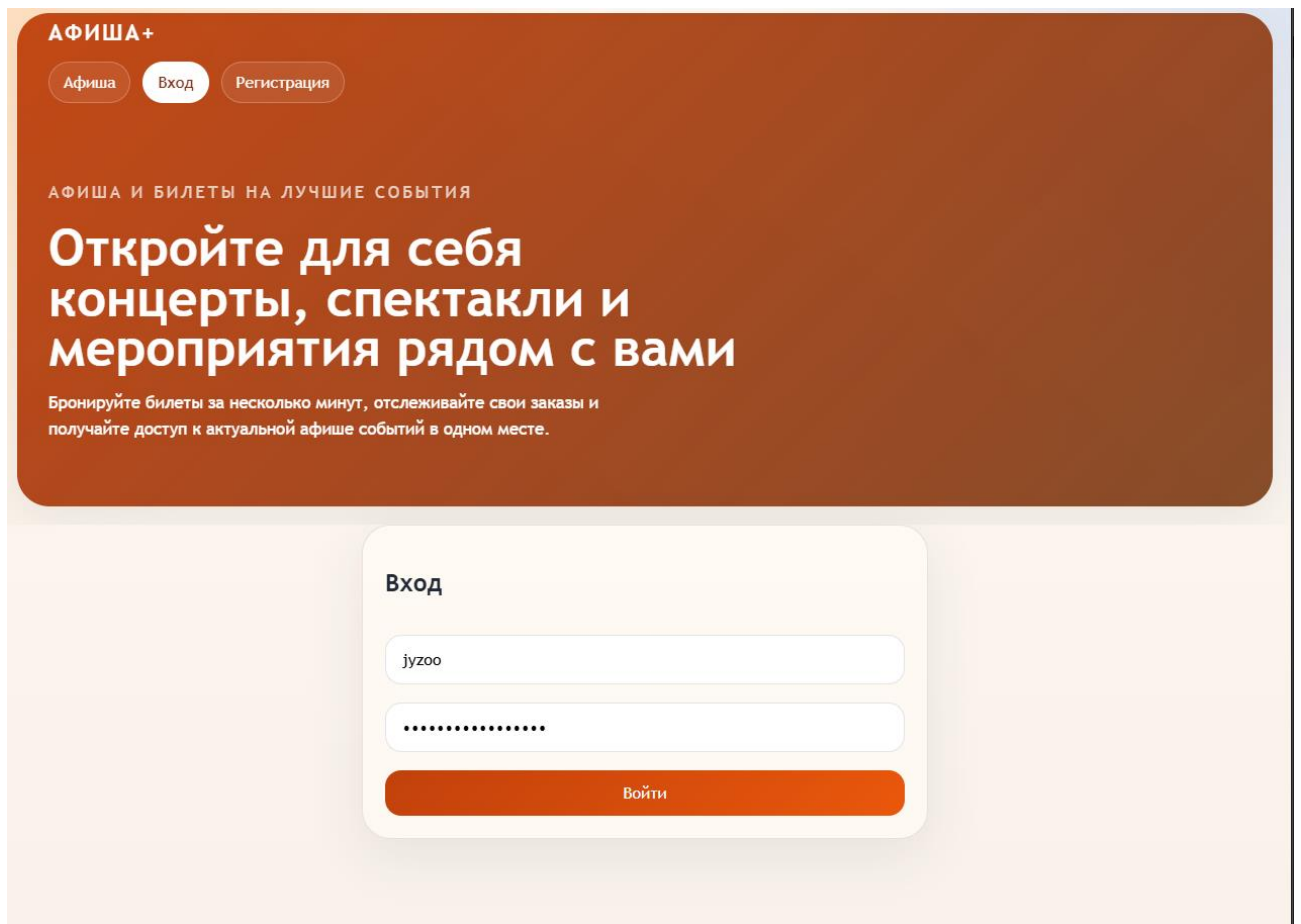
Имя

Фамилия

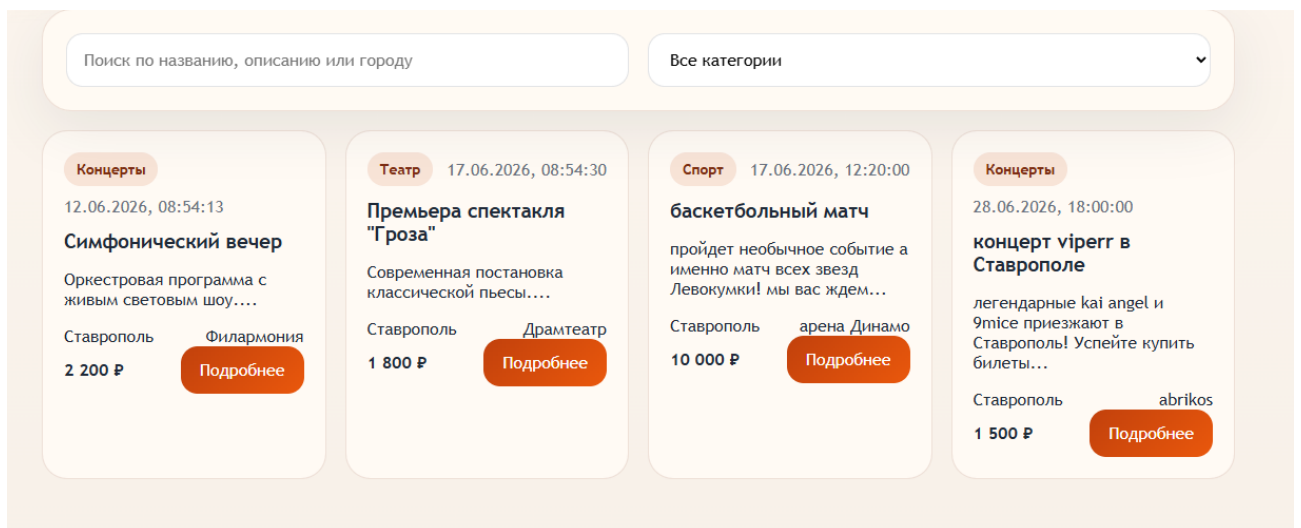
Телефон

Создать аккаунт

Б.3 Страница авторизации



Б.4 Список мероприятий



Б.5 Страница мероприятия

Концерты

концерт v1perг в Ставрополе
легендарные ka1 angel и 9m1ce приезжают в Ставрополь! Успейте купить билеты

Дата	Локация	Адрес
28.06.2026, 18:00:00	Ставрополь, abrikos	генерала Маргелова

Свободно мест
101

1 500 ₽

Организатор: пашок вайпер

1

Телефон

Имя посетителя

Забронировать

Комментарии

Ваш комментарий

Добавить комментарий

Б.6 Создание брони

1 500 ₽

Организатор: пашок вайпер

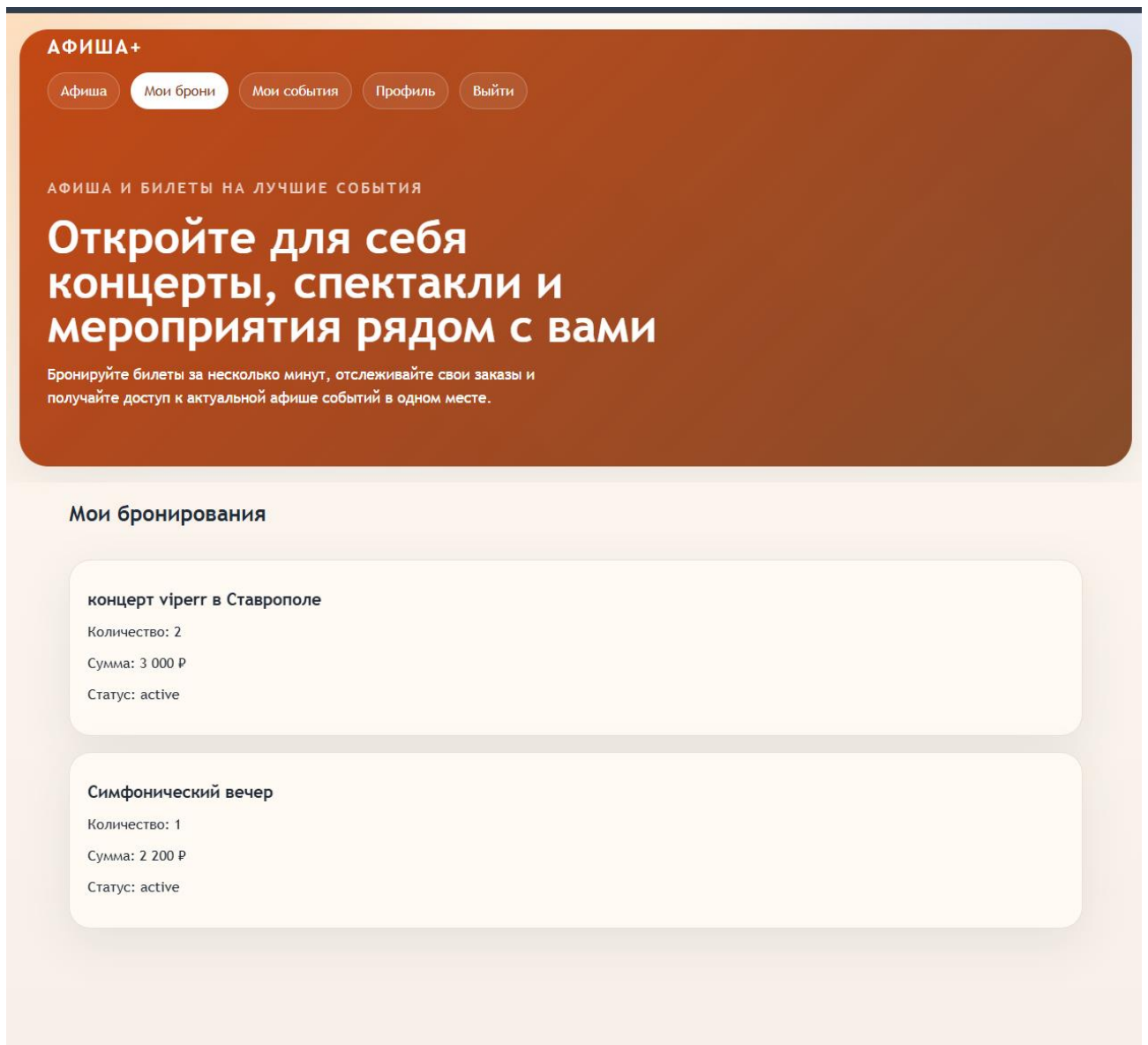
2

+79184348254

Данила Щугарев

Забронировать

Б.7 Личный кабинет пользователя



Б.8 Административная панель Django

